WNDRVR
SUPPORT NETWORK

VXWORKS 7 INTEL ARCHITECTURE GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 Intel Architecture Guide, SR0650

14 September 2020

1. ABOUT THE SUPPORTED INTEL ARCHITECTURES

VxWorks® 7 for Intel supports VxWorks development on 32-bit and 64-bit Intel processors (IA-32 and Intel 64).

While VxWorks on 32-bit processors is capable of addressing 4 GB of memory, on 64-bit processors it supports larger physical and virtual address spaces. VxWorks Intel 64 applications can directly access any valid part of the 64-bit address range.

Note: Several 32-bit Intel processors support more than 32-bits of physical memory and can provide support for somewhat more than 4 GB of RAM. However, applications running on these processors can only access memory within their 32-bit address space and can only access the additional RAM through indirect mechanisms.

WNDRVR

2. INTEL ARCHITECTURE SUPPORT IN VXWORKS 7

1. Supported Processors

This release supports several Intel processor families.

VxWorks 7 has support for 32 and 64-bit processors (IA-32 and Intel 64) on all CPU types except for PENTIUM4, which does not support Intel 64. For more information about your specific processor, refer to your target hardware documentation.

The following table gives a breakdown of the Intel processor support available in this VxWorks 7 release.

Table 1. Intel Processor Support

CPU Type	Processor Cores Supported
PENTIUM4	P7 (Pentium 4)
ATOM	Bonnell (ATOM) cores: Silverthorne, Diamondville, Pineview, Cedarview, Tunnel Creek
CORE	Core
NEHALEM	Nehalem
WESTMERE	Westmere
SANDYBRIDGE	Sandybridge
IVYBRIDGE	Ivybridge
HASWELL	Haswell
BROADWELL	Broadwell
SKYLAKE	Skylake, Kaby Lake and Coffee Lake
SILVERMONT	Silvermont (ATOM) core: Bay Trail
AIRMONT	Airmont (ATOM) cores: Cherry Hill, Braswell
GOLDMONT	Goldmont (ATOM) cores: Apollo Lake, Denverton

WNDRVR

2. Advanced Configuration and Power Interface

Advanced Configuration and Power Interface (ACPI) is an open standard specification for device configuration and hardware power management. ACPI serves as an interface layer between the system firmware (BIOS) and the operating system (OS). Detailed information is available at ACPI Component Architecture project (https://acpica.org/).

VxWorks supports the ACPI specification as defined by the ACPI Component Architecture project. During system startup, the ACPI library performs a device scan and initializes the base ACPI tables. A system level bus (**acpiBus**) is used for attaching ACPI devices such as, PCI (PNP0A03), PCI Express (PNP0A08), and Power Button (PNP0C0C).

VxWorks displays the ACPI tables by calling function acpiTablesShow( ), and displays ACPI Devices by calling function acpiDevicesShow( ).

ACPI Event on page 3 VxWorks processes both fixed and general purpose ACPI events using system control interrupts (SCIs).
Power Button on page 3 VxWorks can support a shutdown system using a power-down button. To enable the power button, a system level power-off needs to be enabled first.

2.1. ACPI Event

VxWorks processes both fixed and general purpose ACPI events using system control interrupts (SCIs).

To process an event, the **acpiEvent** device is attached to the **acpiBus**. To enable a specific event, the ACPI event handler uses ACPI lib. When an ACPI event occurs, it is checked and dispatched in the SCI ISR handler and a specific event is invoked. ACPI events are enabled by adding the **DRV_IA_ACPI_EVENT** component to the VxWorks project.

Parent topic: Advanced Configuration and Power Interface on page 3

2.2. Power Button

VxWorks can support a shutdown system using a power-down button. To enable the power button, a system level power-off needs to be enabled first.

The power off process is enabled by configuring _WRS_CONFIG_SYS_PWR_OFF in the VxWorks project, then the power button driver can be enabled by adding the **DRV_IA_POWER_BUTTON** component to the VxWorks project and the power button handler is registered with ACPI lib. When the power button is pressed, the power button handler is called first, then system level power off is invoked, and finally an ACPI system power state transferring to Soft Off (S5 state in ACPI specification) is called. VxWorks supports dynamically disabling and enabling the power button by calling function powerButtonEnable( ), powerButtonDisable( ) and power button state display using powerButtonShow( ).

Parent topic: Advanced Configuration and Power Interface on page 3

3. ILP32 and LP64 Data Models

The traditional data model issued on 32-bit processors and supported on VxWorks is the ILP32 data model (where ‘I’ means integer, ‘L’ means long, and ‘P’ means pointer).

The value 32 refers to the 32-bit size of those three data types. The 64-bit data model selected for the VxWorks 64-bit platform is the LP64 data model. The value 64 refers to the 64-bit size of the long and pointer data types.

Those two data models are described in the following table and indicate the number of bits representing each data type for both data models.

Table 1.

Type	ILP32 Data Model (Bits)	LP64 Data Model (Bits)
char	8	8
short	16	16
int	32	32
long	32	64
long long	64	64
pointer	32	64
off_t	32 / 64	64
size_t	32	64
float	32	32
double	64	64
long_double	Depends[1] on page 4	Depends
enum	32	32

Because the size of the char, short, and int data types do not differ between the two data models, porting applications developed for a 32-bit environment is straightforward.

The pointer type does change size. You can no longer assume that an int type variable can hold a pointer or any address.

Also note:

- The **long long** type comes from C99. In the LP64 data model, it does not differ from the long data type; however, it does in the ILP32 data model.
- The **off_t** and **size_t** types are POSIX types that represent an offset and a size, respectively. They have been introduced for the data size neutrality of some APIs (that is, you do not have to know what size of objects those types represent in order to use them).
- For the ILP32 data model, the **off_t** type is indicated in the table as having either a 32-bit size or a 64-bit size. This is specific to VxWorks and indicates the size of **off_t** in the kernel environment (32 bits) and in the RTP environment (64 bits already).
- The **long double** type is another C99 type. Its size is not defined by the standard other than being at least equal to that of the double type. For the ILP32 data model, GNU compilers implement the type as a 64-bit type on all architectures except on IA where it is a 96-bit (12 bytes) **long** type. For the LP64 data model, GCC 3.x implements it as a 64-bit type except on IA where it is a 128-bit type. However, GCC 4.x implements it as a 128-bit type on all architectures.

[1] on page 4

The **long double** type is known as a "problematic type" due to both the architectural and compiler-related variation in its implementation. If your code is using this type you must consider carefully whether your code needs to use the **long double** type. If it does, you must check what the compiler is producing for each architecture.

4. Boot Loader Information

In the current release of VxWorks 7, VxWorks image project (VIP) bundles are not supported for the **itl_generic** BSP.

The bundles for boot applications, reboot modes, and board bundles are provided for backward compatibility with existing VxWorks 7 BSPs (not **itl_generic**) only. New development should use the equivalent kernel components (for example, **INCLUDE_FAST_REBOOT** in place of **BUNDLE_MSB_FAST_REBOOT**).

For information specific to your target hardware, see the *Intel® itl_generic BSP Supplement Guide*, Introducing the itl_generic BSP.

Note: The 32-bit support for the **itl_generic** BSP uses a VxWorks 7-style overlapped memory model. Therefore, boot images created outside the **itl_generic** BSP are unable to load an itl_generic-created 32-bit kernel image.

Boot Devices

When your hardware is powered on, the system BIOS code loads and executes the bootstrap loader.

The bootstrap loader is written in 8088 16-bit assembly language. The BIOS obtains the bootstrap loader from the master boot record, which may be in one of several locations: a USB 2.0 key, a hard disk, or other location. When the BIOS finds the bootstrap loader, it transfers execution to it. The bootstrap loader finds the **bootrom.sys** file, loads it into memory, and transfers execution to **romInit**.

Note: For more information on available boot loaders (including multi-stage, UEFI, SYSLINUX, and GRUB) and how to use them, see the*VxWorks 7 Boot Loader User's Guide*, Boot Loader Overview and your BSP *Intel® itl_generic BSP Supplement Guide*, Introducing the itl_generic BSP.

5. Hyperthreading Support

VxWorks 7 supports hyper-threading/simultaneous multi threading (SMT) technology for symmetric multiprocessing (SMP).

A hyper-threaded core appears as two logical processors to the operating system, allowing the OS to schedule two threads simultaneously. These logical processors have independent general register sets and control register sets in the core hardware. The logical processors also share a single hardware execution resource, through which all logical processor instructions must pass. Because this execution resource is shared, it can sometimes act as a bottleneck within the hyper-threaded core.

After enabling hyper-threading in your BIOS setup utility, you may need to add hyper-threading support to your project by adding the **INCLUDE_SMP_SCHED_SMT_POLICY** component provided by VxWorks. This component enables the Simultaneous multi-threading (SMT) scheduling policy and the detection of SMT cores in the OS level. This component is enabled by default for the **itl_generic** BSP but disabled for others. It should not be enabled if the processor doesn't support HT or is disabled in the BIOS. Use the Workbench 4 kernel configuration editor or the wrtool command-line facility to add or remove this functionality in your project.

For information on optimizing the performance of a hyper-threaded core, see the SMP scheduler for SMT information in the *VxWorks 7 Application Programmer's Guide: VxWorks SMP*.

VxWorks also provides the following functions to view and verify the topology information:

vxCpuIdTopology( )

This function calculates and displays the current logical processor topology; it gathers topology information directly from the cores.

vxCpuIdShowTopology( )

This function displays the logical processor topology for the structure pointer passed in.

vxCpuIdMapTopology( )

This function maps the CPU topology using the provided pointer to a topology data structure, and a pointer to a valid MP or APIC data structure; it is called automatically when the BSP is configured for SMP.

6. Hyperthreading and Performance Optimization

It is important to include the component **INCLUDE_CPU_LOW_PWR_IDLE** into a VxWorks image project (VIP) that is intended for an SMT enabled Intel board. This component not only helps to conserve power on the board, it also contributes to optimized hyper-threaded core execution.

Note:

SMT is enabled in VxWorks by:

1. enabling SMT in the target BIOS.
2. adding the component **INCLUDE_SMT_ENABLE** to the VIP.

When **INCLUDE_CPU_LOW_PWR_IDLE** is added to the VIP, VxWorks makes sure that an idle logical processor on the Intel board executes the **HLT** instruction. Assuming a hyper-threaded core is composed of logical processors A and B, when A executes the **HLT** instruction, it goes into an idle state and therefore ceases to use the hyper-threaded core shared execution resource. Consequently, logical processor B has sole access to the execution resource while logical processor A is idling. The hyper-threaded core is therefore executing more effeciently.

When **INCLUDE_CPU_LOW_PWR_IDLE** is removed from the VIP, then an idle logical processor does not enter an idle state. Instead, it continues to actively execute instructions in a **while (1)** loop until the logical processor is given something else to do. Because logical processor A is executing a **while (1)** loop rather than idling, it is therefore still consuming the hyper-threaded core's shared execution resource. Logical processor A is consuming system resources without doing anything useful. Consequently, logical processor B is prohibited from gaining sole access to the shared execution resource. This hyper-threaded core is therefore executing inefficiently.

In an SMT enabled system, the inclusion of **INCLUDE_CPU_LOW_PWR_IDLE** therefore contributes to the optimization of logical processor resources. It places an unused logical processor in an idle mode, allowing the other logical processor to operate more freely than before, making efficient use of the hyper-threaded core.

Removal of **INCLUDE_CPU_LOW_PWR_IDLE** will therefore lead to degradation of performance on SMT enabled systems.

7. Optimized Libraries

Most VxWorks libraries are compiled from portable C source code, but there are some libraries that are compiled from assembly language for better performance.

The following libraries are compiled from assembly language and are optimized for IA-32 and Intel 64 targets:

bLib

buffer manipulation library (including the swab( ) function)

dllLib

doubly-linked list manipulation library

WNDRVR

sllLib

singly-linked list manipulation library

ffsLib

find first bit set library

qPriBMapLib

bit-mapped priority queue library

kernelLockLib

kernel lock library

spinLockIsrNdALib

non-deterministic ISR spinlocks for i86

workQLib

workQ functions

8. Cache Quality of Service

The cache quality of service (QoS) feature enables the use of Intel Cache Allocation Technology (CAT) as part of the Intel Resource Director Technology (RDT) feature set.

Intel CAT provides a method of configuring level 2 (L2) and level 3 (L3) cache allocation to reserve a percentage of the cache for individual CPUs or specific tasks in a symmetric multiprocessing (SMP) system. This allocation reduces cache contention and can prevent compute-intensive tasks from using excessive amounts of cache. For more information on Intel CAT, see the Intel Resource Director Technology (Intel RDT) Allocation Features section of *Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 3B*.

While the L2 and L3 caches are unified caches by default, the Code and Data Prioritization (CDP) extension of Intel CAT allows separate allocations of L3 cache for code and data.

The cache QoS feature allows you to allocate cache by either configuring VxWorks component parameters or calling cache QoS functions.

Implementation of the cache QoS feature requires a processor with Intel CAT support for the cache level that you want to configure —for more information, see *Introduction to Cache Allocation Technology in the Intel® Xeon® Processor E5 v4 Family*.

Cache Allocation

The L2 and L3 caches can be divided by percentage into defined, non-overlapping portions of cache called classes of service (COSs). Note that in Intel documentation, classes of service are refered to as CLOSs. Processors that support the cache QoS feature provide a set of model-specific registers (MSRs) to support COSs, and each COS can be allocated to available CPUs.

Configuring the cache QoS parameters in a VIP allocates COSs to a maximum of eight CPUs.

Calling cache QoS functions at run-time can be used to create COSs and set the COS for more than eight CPUs. This method of cache allocation also allows you to allocate separate portions of cache for code and data.

Note that cache allocation provides write permission, but all CPUs continue to have read permission.

Task-Based Cache Allocation

By default, allocating COSs to CPUs does not prevent tasks from migrating between CPUs and accessing different portions of cache. To ensure that a task is only executed on a specific CPU and is restricted to that portion of cache, you must assign the task to that CPU using task CPU affinity. For more information on setting CPU affinity, see *VxWorks 7 Application Programmer's Guide: CPU Affinity*.

Configuring VxWorks with Cache QoS Support on page 8 Create and configure your VSB and VIP projects to enable the cache QoS component and allocate cache to specific CPUs. You can also restrict a task to run on a specific CPU.
Cache QoS Component and Parameters on page 9 The cache QoS component and parameters allow you to configure a VIP to allocate cache to individual CPUs by percentage.
Cache QoS Functions on page 9 Use cache QoS functions to allocate cache among CPUs up to and above eight in number and to make separate allocations of L2 or L3 cache for code and data with the Intel Code and Data Prioritization (CDP) extension.

8.1. Configuring VxWorks with Cache QoS Support

Create and configure your VSB and VIP projects to enable the cache QoS component and allocate cache to specific CPUs. You can also restrict a task to run on a specific CPU.

Before You Begin

Verify that your target processor supports the cache QoS feature for the cache level that you want to configure by executing the vxCpuIdShow( ) command in the kernel shell.

In the vxCpuIdShow( ) output, an **X** next to the L2 or L3 cache allocation technology option indicates that the cache QoS feature is supported for that cache level.

Procedure

1. Create and build a VSB based on the **itl_generic** BSP.
2. Create a VIP based on the VSB and add the **INCLUDE_CACHE_QOS** component to the VIP.
3. To allocate cache use to a specific CPU, set its associated parameter value as a portion of 100. For more information, see Cache QoS Component and Parameters on page 9.

   Any configured parameter values must add up to 100.

   Any CPU that you do not configure maintains the default setting of 100% cache allocation, able to read from and write to the entire cache.

   > Note: If you want to allocate cache to more than eight CPUs or allocate separate cache portions for code and data, add the **cacheQoSLib** library to a file that will be included in the VIP build and create and allocate classes of service (COSs) with the cache QoS functions. For more information, see Cache QoS Functions on page 9.

4. If you want to restrict a task to run on a specific CPU, set the CPU affinity of the task.

   For more information on setting CPU affinity, see *VxWorks 7 Application Programmer's Guide* CPU Affinity.

5. Build the VIP.

Parent topic: Cache Quality of Service on page 7

8.2. Cache QoS Component and Parameters

The cache QoS component and parameters allow you to configure a VIP to allocate cache to individual CPUs by percentage.

Components	Description
INCLUDE_CACHE_QOS	Permits cache allocation per CPU in the VIP

The cache QoS feature includes configurable parameters to allocate L2 and L3 cache for each CPU.

Parameters	Description	Permissible Values
L2_CACHE_ALLOC_CPU*NUM*	Determines the portion of cache assigned to CPU*NUM*, where *NUM* is the CPU number between 0 and 7	The value for this parameter is a representation of the percentage of cache being allocated to the CPU as a portion of 100. Any configured parameter values must add up to 100. CPUs left with the default value of 100 will have access to write to and read from the entire L2 cache. For example, four CPUs with varied cache allocation could be defined as follows: L2_CACHE_ALLOC_CPU0 **15** L2_CACHE_ALLOC_CPU1 **15** L2_CACHE_ALLOC_CPU2 **30** L2_CACHE_ALLOC_CPU3 **40**
L3_CACHE_ALLOC_CPU*NUM*	Determines the portion of cache assigned to CPU*NUM*, where *NUM* is the CPU number between 0 and 7	The value for this parameter is a representation of the percentage of cache being allocated to the CPU as a portion of 100. Any configured parameter values must add up to 100. CPUs left with the default value of 100 will have access to write to and read from the entire L3 cache. For example, four CPUs with equal cache allocation would be defined as follows: L3_CACHE_ALLOC_CPU0 **25** L3_CACHE_ALLOC_CPU1 **25** L3_CACHE_ALLOC_CPU2 **25** L3_CACHE_ALLOC_CPU3 **25**

Parent topic: Cache Quality of Service on page 7

8.3. Cache QoS Functions

Use cache QoS functions to allocate cache among CPUs up to and above eight in number and to make separate allocations of L2 or L3 cache for code and data with the Intel Code and Data Prioritization (CDP) extension.

The L2 and L3 caches can be divided by percentage into defined, non-overlapping portions of cache called classes of service (COSs). Processors that support the cache QoS feature provide a set of model-specific registers (MSRs) to support COSs, and each COS can be allocated to available CPUs. The number of MSRs provided by the processor determines the maximum number of COSs.

Allocating separate portions of cache for code and data with the CDP extension requires two MSRs for each COS—one MSR for code and one for data; therefore, operating with CDP reduces the number of possible COS settings by half.

To determine the number of MSRs provided by your processor, execute the vxCpuIdShow( ) command in the kernel shell.

Function	Description
cacheQosShow( )	Prints cache QoS status and settings to the console, including the current COS level definitions, the CPU COS index, and whether CDP is enabled
cacheQosInit( )	Initializes the cache QoS library
cacheQosClassCreate( )	Creates and initializes a class of service (COS) ID and sets the appropriate MSR register in the CPUs indicated in cpuset
cacheQosClassModify( )	Modifies a COS using a fractional representation
cacheQosBitmaskClassCreate( )	Creates and initializes a COS ID using explicit bit mask allocation and sets the appropriate MSR register in the CPUs indicated in cpuset
cacheQosBitmaskClassModify( )	Modifies an existing COS ID to set a different bitmask or add an allocation for a different cache level
cacheQosClassDelete( )	Removes the COS ID and restores the associated registers to defaults
cacheQosClassSet( )	Sets the COS ID in the current CPU
cacheQosClassCpuSet( )	Sets the COS for the CPUs in the cpuset
cacheQosCdpEnable( )	Enables the Code and Data Prioritization (CDP) extension
cacheQosCdpDisable( )	Disables the CDP extension

For more information, see the **cacheQoSLib** entry in *API Reference: Kernel Core OS*.

Parent topic: Cache Quality of Service on page 7

9. Supported Functions in mathALib

For IA-32 and Intel 64 targets, various double-precision floating-point functions are supported.

WNDRVR

acos( )	**asin( )**	**atan( )**	**atan2( )**	**ceil( )**	**cos( )**
cosh( )	**exp( )**	**fabs( )**	**floor( )**	**fmod( )**	**infinity( )**
irint( )	**iround( )**	**log( )**	**log10( )**	**log2( )**	**pow( )**
round ( )	**sin( )**	**sincos( )**	**sinh( )**	**sqrt( )**	**tan( )**
tanh( )	**trunc( )**				

The corresponding single-precision floating-point functions are not supported. In this release, hyperbolic cosine, sine, and tangent functions are supported. For more information, see the reference entry for **mathALib** and the individual reference entries for each function.

Under the 64-bit implementation, floating-point values are passed between the functions and subroutines using the XMM registers. The XMM registers are preserved across function/subroutine calls. The **FXSAVE**/**FXRSTR** functions save and restore the contents of registers **XMM[0]** through **XXM[15]**.

10. Implicit Vectorization and FPU-based Compiler Optimizations

The Clang (LLVM) compiler is capable of optimizing the code it produces, performing certain data-intensive operations using vector and/or floating point instructions, even when the application code is not written explicitly to imply this intent.

By default, VxWorks uses the -mno-implicit-float compiler option to suppress the generation of these code optimizations.

However, you can reenable the generation of these code optimizations by removing the option -mno-implicit-float. When you do this, you must take care to spawn any VxWorks tasks that will by optimized by the compiler to use floating point or SIMD registers with the appropriate task option (VX_FP_TASK).

For detailed information about the compiler and library changes, see the *VxWorks 7 C11, C++14, and Compiler Migration Reference*.

Resolve Unexpected Math Problems on page 11

10.1. Resolve Unexpected Math Problems

In some scenarios, the VxWorks Clang/LLVM compiler will generate float instructions during compilation. If your application task's math operations do not appear to generate consistently correct results, it is likely that the processor floating point registers are being used by your application task and then unexpectedly overwritten again by other tasks when the kernel reschedules them.

For example, the following scenario will cause the Clang/LLVM compiler to generate float instructions:

- 64-bit VxWorks on Intel architecture (not 32-bit).
- declaring a function to be variadic. For example:

```
int add_up_my_parameters (int count,…)
```

- compiling the application with the option -mno-implicit-float defined.
- turning off compiler optimizations.

To resolve the situation where your function fails to generate consistently correct math results, you must adjust the taskSpawn( ) call for the VxWorks task encapsulating the erroneous function. Add the option **VX_FP_TASK** as a task option. This will ensure that, if the

floating point registers are being used by your application task, they will be protected when the kernel reschedules tasks. The function in this application task will then generate consistently correct math results.

Parent topic: Implicit Vectorization and FPU-based Compiler Optimizations on page 11

11. Architecture-Specific Global Variables Reference

The **sysLib.c** and **sysALib.s** files contain the global variables shown in the table below.

Table 1. Architecture-Specific Global Variables

Global Variable	Value	Description
sysCsSuper	0x08	Code selector for the supervisor mode task.
sysCsExe	0x18	Code selector for exceptions.
sysCsInt	0x20	Code selector for interrupts.
sysIntIdtType	0x0000ef00 = trap gate 0x0000ee00 (default) = interrupt gate	This variable is used when VxWorks initializes the interrupt vector table. The choice of trap gate versus interrupt gate affects all interrupts (vectors 0x20 through 0xff). Note that the required choice for an SMP image is an interrupt gate.
sysGdt[]	0xffff limit (default)	The global descriptor table begins with five entries. The first is a null descriptor. The second and third are for task-level functions. The fourth is for exceptions. The fifth is for interrupt-level functions. If kernel hardening is enabled, additional entries are added for task gate management of the OSM stack.
sysProcessor	Extracted by the CPUID instruction	The processor type (kept for backward compatibility).
vxProcessor	Extracted by the CPUID instruction	Set by the **vxCpuIdProbe( )** function.
sysCoprocessor	0 = no coprocessor 1 = 387 coprocessor 2 = 487 coprocessor	The type of floating-point coprocessor (set by the VxWorks **fppProbe( )** function).
sysCpuId	CPUID structure	Dynamically obtained processor identification and supported features (set by the **vxCpuIdProbe( )** function).

12. Architecture-Specific Functions Reference

fppArchSwitchHookEnable( )

The function header for this function is as follows:

```
STATUS fppArchSwitchHookEnable
  (BOOL enable)
```

This function enables or disables the architecture-specific FPU switch hook function that detects illegal FPU/MMX usage.

fppCtxShow( )

The function header for this function is as follows:

```
void fppCtxShow
  (FP_CONTEXT * f)
```

This function prints the contents of a task's floating-point register.

fppRegListShow( )

The function header for this function is as follows:

```
void fppRegListShow (void)
```

This function prints a list of available registers.

intStackEnable( )

The function header for this function is as follows:

```
STATUS intStackEnable
  (BOOL enable)
```

This function enables or disables the interrupt stack usage. Set to **TRUE** to enable, **FALSE** to disable.

pentiumBtc( )

The function header for this function is as follows:

```
STATUS pentiumBtc
  (char * pFlag)
```

This function executes an atomic compare-and-exchange instruction to clear a bit.

pentiumBts( )

The function header for this function is as follows:

```
STATUS pentiumBts
  (char * pFlag)
```

This function executes an atomic compare-and-exchange instruction to set a bit.

pentiumMcaEnable( )

The function header for this function is as follows:

```
void pentiumMcaEnable
  (BOOL enable)
```

This function enables or disables the machine check architecture (MCA).

pentiumMcaShow( )

The function header for this function is as follows:

```
void pentiumMcaShow (void)
```

This function shows machine check global control registers and error reporting register banks.

pentiumMsrGet( ) (IA-32)

The function header for this function is as follows:

```
void pentiumMsrGet
  (
  int address,
  long long int * pData
  )
```

This function gets the contents of the specified model-specific register (MSR).

pentiumMsrGet( ) (Intel 64)

The function header for this function is as follows:

```
void pentiumMsrGet
  (
  long addr,    /* MSR address */
  long * pData  /* MSR data */
  )
```

This function gets the contents of the specified model-specific register (MSR).

pentiumMsrInit( )

The function header for this function is as follows:

```
STATUS pentiumMsrInit (void)
```

This function initializes all MSRs.

Note: This function is deprecated for all CPUs except Pentium 4.

pentiumMsrSet( ) (IA-32)

The function header for this function is as follows:

```
void pentiumMsrSet
  (
  int address,
  long long int * pData
  )
```

This function sets the value of the specified MSR.

pentiumMsrSet( ) (Intel 64)

The function header for this function is as follows:

```
void pentiumMsrSet
  (
```


WNDRVR

```
  long addr,    /* MSR address */
  long * pData  /* MSR data */
  )
```

This function sets the value of the specified MSR.

pentiumMsrShow( )

The function header for this function is as follows:

```
void pentiumMsrShow (void)
```

This function shows all MSRs.

pentiumMtrrEnable( )

The function header for this function is as follows:

```
void pentiumMtrrEnable (void)
```

This function enables the memory type range register (MTRR).

pentiumMtrrDisable( )

The function header for this function is as follows:

```
void pentiumMtrrDisable (void)
```

This function disables the MTRR.

pentiumMtrrGet( )

The function header for this function is as follows:

```
void pentiumMtrrGet
  (MTRR * pMtrr)
```

This function gets MTRRs to the MTRR table specified by the pointer.

pentiumMtrrSet( )

The function header for this function is as follows:

```
void pentiumMtrrSet
  (MTRR * pMtrr)
```

This function sets MTRRs from the MTRR table specified by the pointer.

pentiumPmcStart( )

The function header for this function is as follows:

```
STATUS pentiumPmcStart
  (
  int pmcEvtSel0;
  int pmcEvtSel1;
  )
```

This function starts PMC0 and PMC1.

pentiumPmcStart0( ) (IA-32)

The function header for this function is as follows:

```
STATUS pentiumPmcStart0
  (int pmcEvtSel0)
```

This function starts PMC0 only.

pentiumPmcStart1( ) (IA-32)

The function header for this function is as follows:

```
STATUS pentiumPmcStart1
  (int pmcEvtSel1)
```

This function starts PMC1 only.

pentiumPmcStop( )

The function header for this function is as follows:

```
void pentiumPmcStop (void)
```

This function stops PMC0 and PMC1.

pentiumPmcStop0( )

The function header for this function is as follows:

```
void pentiumPmcStop0 (void)
```

This function stops PMC0 only.

pentiumPmcStop1( )

The function header for this function is as follows:

```
void pentiumPmcStop1 (void)
```

This function stops PMC1 only.

pentiumPmcGet( )

The function header for this function is as follows:

```
void pentiumPmcGet
  (
  long long int * pPmc0;
  long long int * pPmc1;
  )
```

This function gets the contents of PMC0 and PMC1.

pentiumPmcGet0( )

The function header for this function is as follows:

```
void pentiumPmcGet0
  (long long int * pPmc0)
```

This function gets the contents of PMC0.

pentiumPmcGet1( )

The function header for this function is as follows:

```
void pentiumPmcGet1
  (long long int * pPmc1)
```

This function gets the contents of PMC1.

pentiumPmcReset( )

The function header for this function is as follows:

```
void pentiumPmcReset (void)
```

This function resets the contents of PMCO and PMC1.

pentiumPmcReset0( )

The function header for this function is as follows:

```
void pentiumPmcReset0 (void)
```

This function resets PMC0.

pentiumPmcReset1( )

The function header for this function is as follows:

```
void pentiumPmcReset1 (void)
```

This function resets PMC1.

pentiumSerialize( )

The function header for this function is as follows:

```
void pentiumSerialize (void)
```

This function serializes by executing the CPUID instruction.

pentiumPmcShow( )

The function header for this function is as follows:

```
void pentiumPmcShow
  (BOOL zap)
```

This function shows PMC0 and PMC1, and resets them if the parameter zap is **TRUE**.

pentiumTlbFlush( )

The function header for this function is as follows:

```
void pentiumTlbFlush (void)
```

This function flushes the translation lookaside buffers (TLBs).

pentiumTscReset( )

The function header for this function is as follows:

```
void pentiumTscReset (void)
```

This function resets the timestamp counter (TSC).

pentiumTscGet32( )

The function header for this function is as follows:

```
UINT32 pentiumTscGet32 (void)
```

This function gets the lower half of the 64-bit TSC.

pentiumTscGet64( )

The function header for this function is as follows:

```
void pentiumTscGet64
  (long long int * pTsc)
```

This function gets the 64-bit TSC.

sysCpuProbe( )

The function header for this function is as follows:

```
UINT sysCpuProbe (void)
```

This function gets information about the CPU with CPUID.

sysInByte( )

The function header for this function is as follows:

```
UCHAR sysInByte
  (int port)
```

This function reads one byte from I/O.

sysInWord( )

The function header for this function is as follows:

```
USHORT sysInWord
  (int port)
```

This function reads one word (two bytes) from I/O.

sysInLong( ) (IA-32)

The function header for this function is as follows:

```
ULONG sysInLong
  (int port)
```

This function reads one long word (four bytes) from I/O.

sysInLong( ) (Intel 64)

The function header for this function is as follows:

```
UINT32 sysInLong (int port)
```

This function reads one long word (four bytes) from I/O.

sysOutByte( )

The function header for this function is as follows:

```
void sysOutByte
  (int port, char data)
```

This function writes one byte to I/O.

sysOutWord( )

The function header for this function is as follows:

```
void sysOutWord
  (int port, short data)
```

This function writes one word (two bytes) to I/O.

sysOutLong( ) (IA-32)

The function header for this function is as follows:

```
void sysOutLong
  (int port, long data)
```

This function writes one long word (four bytes) to I/O.

sysOutLong( ) (Intel 64)

The function header for this function is as follows:

```
void sysOutLong
  (int port, int data)
```

This function writes one long word (four bytes) to I/O.

sysInWordString( )

The function header for this function is as follows:

```
void sysInWordString
  (
  int port,
  short *address,
  int count
  )
```

This function reads a word string from I/O.

sysInLongString( )

The function header for this function is as follows:

```
void sysInLongString
  (
  int port,
  long *address,
  int count
  )
```

This function reads a long string from I/O.

sysOutWordString( )

The function header for this function is as follows:

```
void sysOutWordString
  (
  int port,
  short *address,
  int count
  )
```

This function writes a word string to I/O.

sysOutLongString( )

The function header for this function is as follows:

```
void sysOutLongString
  (
  int port,
  long *address,
  int count
  )
```

This function writes a long string to I/O.

sysDelay( )

The function header for this function is as follows:

```
void sysDelay (void)
```

This function allows enough recovery time for port accesses.

sysIntDisablePIC( )

The function header for this function is as follows:

```
STATUS sysIntDisablePIC
  (int intLevel)
```

This function disables a programmable interrupt controller (PIC) interrupt level.

sysIntEnablePIC( )

The function header for this function is as follows:

```
STATUS sysIntEnablePIC
  (int intLevel)
```

This function enables a PIC interrupt level.

sysOSMTaskGateInit( )

The function header for this function is as follows:

```
STATUS sysOSMTaskGateInit (void)
```

This function initializes the OSM stack. (See Operating System Miscue (OSM) Stack on page 43).

vxCpuIdAdd( ) (IA-32)

The function header for this function is as follows:

```
STATUS vxCpuIdAdd(VX_CPUID_ENTRY *entry)
```

This function adds a CPUID signature to the CPUID signature table. Use this function to add support for new processors not yet defined in the global CPUID signature table. Each entry contains a CPUID signature and corresponding processor family to which the CPU belongs.

vxCpuIdLibInit( ) (IA-32)

The function header for this function is as follows:

```
void vxCpuIdLibInit(void)
```

This function initializes the CPUID library.

vxCpuIdOverride( ) (IA-32)

The function header for this function is as follows:

```
STATUS vxCpuIdOverride(VX_CPUID *cpuid, VX_CPUID_OVERRIDE *entries, int cnt)
```

This function allows a BSP to override any subset of feature flags obtained from vxCpuIdProbe( ). An array of override entries and count of entries is passed into the function. Each entry can either enable or disable a feature.

vxCpuIdProbe( ) (IA-32)

The function header for this function is as follows:

```
int vxCpuIdProbe(VX_CPUID *)
```

This function executes the **CPUID** instruction and populates the global **VX_CPUID** structure.

vxCpuIdShow( ) (IA-32)

The function header for this function is as follows:

```
vxCpuIdShow(VX_CPUID *)
```

This function prints all output from the **CPUID** probe and all overrides from the vxCpuIdOverride( ) function.

vxCpuShow( )

The function header for this function is as follows:

```
void vxCpuShow (void)
```

This function shows CPU type, family, model, and supported features.

This function has been deprecated. Use vxCpuIdShow( ) instead to show CPU information.

vxCr[0234]Get( )

The function header for this function is as follows:

```
int vxCr[0234]Get (void)
```

This function gets respective control register content.

vxCr[0234]Set( )

The function header for this function is as follows:

```
void vxCr[0234]Set (int value)
```

This function sets a value to the respective control register.

vxDrGet( )

The function header for this function is as follows:

```
void vxDrGet
  (
  int * pDr0,
  int * pDr1,
  int * pDr2,
  int * pDr3,
  int * pDr4,
  int * pDr5,
  int * pDr6,
  int * pDr7
  )
```

This function gets debug register content.

vxDrSet( )

The function header for this function is as follows:

```
void vxDrSet
  (
  int dr0,
  int dr1,
  int dr2,
  int dr3,
  int dr4,
  int dr5,
  int dr6,
  int dr7
  )
```

This function sets debug register values.

vxDrShow( )

The function header for this function is as follows:

```
void vxDrShow (void)
```


WNDRVR

This function shows the debug registers.

vxEflagsGet( )

The function header for this function is as follows:

```
int vxEflagsGet (void)
```

This function gets the EFLAGS register content.

vxEflagsSet( )

The function header for this function is as follows:

```
void vxEflagsSet (int value)
```

This function sets the value of the EFLAGS register.

vxTssGet( )

The function header for this function is as follows:

```
int vxTssGet (void)
```

This function gets the task register content.

Note: This function is for Wind River internal use only.

vxTssSet( )

The function header for this function is as follows:

```
void vxTssSet (int value)
```

This function sets the task register value.

Note: This function is for Wind River internal use only.

vx[GIL]dtrGet( )

The function header for this function is as follows:

```
void vx[GIL]dtrGet
  (long long int * pValue)
```

This function gets the **GDTR**, **IDTR**, and **LDTR** register content, respectively.

vxSseShow( ) (IA-32)

The function header for this function is as follows:

```
void vxSseShow (int taskId)
```

This function prints the contents of a task's Streaming SIMD Extension (SSE) register context, if any, to the standard output device.

vxSseShow( ) (Intel 64)

The function header for this function is as follows:

WNDRVR

```
void vxSseShow (long taskId)
```

This function prints the contents of a task's Streaming SIMD Extension (SSE) register context, if any, to the standard output device.

Atomic Operators for Intel 64 on page 24 Support for Intel 64-based processors introduces a number of new fixed-size atomic operator APIs to handle the various scenarios in LP64.
Register Functions for IA-32 on page 25
Register Functions and Usage for Intel 64 Reference on page 26
Breakpoints and the bh( ) Function on page 30 VxWorks for Intel Architecture supports both software and hardware breakpoints.
Disassembler: l( ) on page 31 If you are using the target shell, the VxWorks disassembler l( ) function does not support 16-bit code compiled for earlier generations of 80x86 processors.
Memory Probe: vxMemProbe( ) on page 31
Interrupt Lock Level: intLock( ) and intUnlock( ) on page 32 The Intel architecture includes a single interrupt signal for external interrupts, and is able to enable and disable external interrupts to the CPU.
Interrupt Lock Level: intCpuLock( ) and intCpuUnlock( ) on page 32 The Intel architecture includes a single interrupt signal for external interrupts, and is able to enable and disable external interrupts to individual CPU cores.
IntArchLib: intVecSet2( ) and intVecGet2( ) on page 32 The functions intVecSet2( ) and intVecGet2( ) replace intVecSet( ) and intVecGet( ), respectively.
pentiumLib, pentiumALib, and pentiumShow: pentiumXXX( ) on page 32 Functions that manipulate the memory type range registers (MTRR), performance monitoring counter (PMC), timestamp counter (TSC), machine check architecture (MCA), and model specific registers (MSR) are included.
vxLib, vxALib, and vxShow: vxXXX( ) on page 33 The vxCpuShow( ) function shows the CPU type, family, model, and supported features.
taskSRSet( ) on page 33 The taskSRSet( ) function sets its second parameter to the **EFLAGS** register of the specified task.

12.1. Atomic Operators for Intel 64

Support for Intel 64-based processors introduces a number of new fixed-size atomic operator APIs to handle the various scenarios in LP64.

Table 1 on page 24 shows the function changes from previous releases of VxWorks and Table 2 on page 25 shows what they do.

Table 1. New Atomic Operators

New	Old
vxAtomicAdd( )	**AtomicAdd( )**
vxAtomicAnd( )	**AtomicAnd( )**
vxAtomicClear( )	**AtomicClear( )**
vxAtomicDec( )	**AtomicDec( )**
vxAtomicGet( )	**AtomicGet( )**
vxAtomicInc( )	**AtomicInc( )**
vxAtomicNand( )	**AtomicNand( )**
vxAtomicOr( )	**AtomicOr( )**

New	Old
vxAtomicSet( )	**AtomicSet( )**
vxAtomicSub( )	**AtomicSub( )**
vxAtomicXor( )	**AtomicXor( )**
vxAtomicCas( )	**AtomicCas( )**

Table 2. Atomic Functions and Parameter Sizes

Function	Parameter Size (ILP32/LP64)
vxAtomicXxx( )	**atomic_t( )** (32/64)
vxAtomic32Xxx( )	**atomic32_t( )** (32/32)
vxAtomic64Xxx( )	atomic64_t( ) (64[1] on page 4/64)

The vxAtomic64Xxx( ) function is available in ILP32. However, the use of these functions is subject to the availability of such an operation in the target architecture.

Do not use these functions on architectures that do not support 64-bit atomic operators or a link error occurs.

The vxTas( ) function is deprecated in favor of vxAtomicCas( ).

The vxCas( ) function is available as an alias in 32-bit configurations for backward compatibility, and is deprecated. Under LP64 only the vxAtomicCas( ) variants are available.

Register Functions for IA-32

The following functions read IA-32 register values, and require one parameter, the task ID:

Parent topic: Architecture-Specific Functions Reference on page 12

[1] on page 4

The architecture must support 64-bit atomic operations.

12.2. Register Functions for IA-32

The following functions read IA-32 register values, and require one parameter, the task ID:

- eax( )
- ebx( )
- ecx( )
- edx( )
- esi( )
- ebp( )
- esp( )

WNDRVR

- eflags( )

Parent topic: Architecture-Specific Functions Reference on page 12

12.3. Register Functions and Usage for Intel 64 Reference

The following tables detail the 64-bit register set compared with the 32-bit register set

The register set for Intel 64 is: **%rax**, **%rbx**, **%rcx**, **%rdx**, **%rsi**, **%rdi**, **%rbp**, **%rsp**, **%r8-%r15**, **%rip**, **%eflags**, **%cs**, **%ss**, **%ds**, **%es**, **%fs**, **%gs**.

The following functions read Intel 64 register values, and require one parameter, the task ID:

- rdi( )
- rsi( )
- rbp( )
- rsp( )
- rbx( )
- rdx( )
- rcx( )
- rax( )
- rflags( )
- r8( )
- r9( )
- r10( )
- r11( )
- r12( )
- r13( )
- r14( )
- r15( )

Expanded Intel 64 Register Set

Register	Use	Preserved Across Function Calls (Yes/No)
%rax	Temporary register. With variable arguments passes information about the number of SSE registers used. 1st return register.	No
%rbx	Callee-saved register. Optionally used as basepointer.	Yes
%rcx	Used to pass 4th integer argument to functions.	No
%rdx	Used to pass 3rd argument to functions; 2nd return register.	No
%rsp	Stack pointer.	Yes

Register	Use	Preserved Across Function Calls (Yes/No)
%rbp	Callee-saved register. VxWorks uses as framepointer.	Yes
%rsi	Used to pass 2nd argument to functions.	No
%rdi	Used to pass 1st argument to functions.	No
%r8	Used to pass 5th argument to functions.	No
%r9	Used to pass 6th argument to functions.	No
%r10	Temporary register. Used for passing a function's static chain pointer.	No
%r11	Temporary register.	No
%r12 - %r15	Callee-saved registers.	Yes
%xmm0 - %xmm1	Used to pass and return floating-point arguments.	No
%xmm2 - %xmm7	Used to pass floating-point arguments.	No
%xmm8 - %xmm15	Temporary registers.	No
%mmx0 - %mmx7	Temporary registers.	No
%st0 - %st1	Temporary registers. Used to return long double arguments.	No
%st2 - %st7	Temporary registers.	No
%fs	Reserved for system to keep track of TLS (thread local storage as thread-specific data register) – currently not used in VxWorks for 32-bit architectures.	No
mxcsr	SSE2 control and status word.	Partial (specific control status bits); refer to your Intel documentation for details.
X87 SW	x87 status word.	Yes
X87 CW	x87 control word.	Yes
%gs	Store the address of kernel cpu var.	No

Arguments Passed by Function Calls

The following table identifies the registers that the user must save and restore across functions calls.

Register		Use	Preserved Across Function Calls (Yes/No)
Long	**Int**		
%rdi	**%edi**	1st argument to functions	No
%rsi	**%esi**	2nd argument to functions	No
%rdx	**%edx**	3rd argument to functions	No
%rcx	**%ecx**	4th argument to functions	No
%r8	**%r8d**	5th argument to functions	No
%r9	**%r9d**	6th argument to functions	No
%sp	**+ SP_ARG7**	7th argument to functions	On stack
%sp	**+ SP_ARG8**	8th argument to functions	On stack
%sp	**+ SP_ARG9**	9th argument to functions	On stack
%sp	**+ SP_ARG10**	10th argument to functions	On stack

Return Values Passed by Function Calls

The following table shows how arguments are passed by function calls.

Register	Use	Preserved Across Function Calls (Yes/No)
%rax	1st return register	No
%rdx	2nd return register	No

Temporary Registers

The following table shows the temporary registers. It is not necessary to save or restore their contents when used within a function. If these registers are used in asynchronously entered functions, they must be saved and restored by the function.

Register	Use	Preserved Across Function Calls (Yes/No)
%rax	Temporary register	No
%r10	Temporary register; used for passing a function's static chain pointer	No
%r11	Temporary register	No

Additional General Purpose Registers

The following table shows the four additional general-purpose registers available, if they are saved and restored within the function using them.

Additional General Purpose Registers

Register	Use	Preserved Across Function Calls (Yes/No)
%r12	Callee-saved register	Yes
%r13	Callee-saved register	Yes
%r14	Callee-saved register	Yes
%r15	Callee-saved register	Yes

Program Flow

The following table shows the control program flow registers. These registers are always accessed by their 64-bit name.

Program Flow Control Registers

Register	Use	Preserved Across Function Calls (Yes/No)
%rsp	Stack pointer	Yes
%rbp	Frame pointer; callee-saved register	Yes
%rip	Instruction Pointer	N/A[1] on page 4

Register Naming and Size

The 32-bit register set can be accessed as 64-bit registers by using the %rXx naming convention. The additional registers are accessed as shown in the following table.

Register Name and Size

Register Name	Size
%ah/al	8 bits
%ax	16 bits
%eax	32 bits
%rax	64 bits

New Registers r8-r15	Size
%rXb (r8b)	8 bits
%rXw (r8w)	16 bits
%rXd (r8d)	32 bits
%rX (r8)	64 bits

Parent topic: Architecture-Specific Functions Reference on page 12

[1] on page 4

Always points to the instruction being executed.

12.4. Breakpoints and the bh( ) Function

VxWorks for Intel Architecture supports both software and hardware breakpoints.

When you set a software breakpoint, VxWorks replaces an instruction with an int 3 software interrupt instruction. VxWorks restores the original code when the breakpoint is removed. The instruction cache is purged each time VxWorks changes an instruction to a software break instruction.

A hardware breakpoint uses the processor's debug registers to set the breakpoint. The Pentium architectures have four breakpoint registers. If you are using the target shell, you can use the bh( ) function to set hardware breakpoints. The function is declared as follows

IA-32:

```
STATUS bh
    (
    INSTR        *addr,        /* where to set breakpoint, or       */
                               /* 0 = display all breakpoints       */
    int          type,         /* breakpoint type; see below        */
    int          task,         /* task to set breakpoint;           */
                               /* 0 = set all tasks                 */
    int          count,        /* number of passes before hit       */
    BOOL         quiet,        /* TRUE = don't print debug info     */
                               /* FALSE = print debug info          */
    )
```

Intel 64:

```
STATUS bh
        (
        INSTR        *addr,         /* where to set breakpoint, or       */
                                                                  /* 0 = display all breakpoint
s       */
        int          type,          /* breakpoint type; see below        */
        long         taskNameOrId,  /* task to set breakpoint;           */
                                                                  /* 0 = set all task
s                 */
        int          count,         /* number of passes before hit       */
```

```
        BOOL         quiet,         /* TRUE = don't print debug info     */
                                                                  /* FALSE = print debug inf
o          */
        )
```

The bh( ) function takes the following types in parameter type:

BRK_INST

Instruction hardware breakpoint (0x00)

BRK_DATAW1

Data write 1-byte breakpoint (0x01)

BRK_DATAW2

Data write 2-byte breakpoint (0x05)

BRK_DATAW4

Data write 4-byte breakpoint (0x0d)

BRK_DATARW1

Data read-write 1-byte breakpoint (0x03)

BRK_DATARW2

Data read-write 2-byte breakpoint (0x07)

BRK_DATARW4

Data read-write 4-byte breakpoint (0x0f)

A maximum number of hardware breakpoints can be set on the target system. This is a hardware limit and cannot be changed. For Intel Architecture targets, this limit is four hardware breakpoints. The address parameter of a hardware breakpoint command does not need to be 4-bytes aligned for data breakpoints on Intel. The address parameter is 1-byte aligned if width access is 1 byte, 2-bytes aligned if width access is 2 bytes, and 4-bytes aligned if width access is 4 bytes.

For more information, see the reference entry for bh( ).

Parent topic: Architecture-Specific Functions Reference on page 12

12.5. Disassembler: l( )

If you are using the target shell, the VxWorks disassembler l( ) function does not support 16-bit code compiled for earlier generations of 80x86 processors.

However, the disassembler does support 32-bit code for IA-32 processors. It also supports 64-bit code for Intel 64 processors.

Parent topic: Architecture-Specific Functions Reference on page 12

12.6. Memory Probe: vxMemProbe( )

The vxMemProbe( ) function, which probes an address for a bus error, is supported on Intel architectures by trapping both general protection faults and page faults.

WNDRVR

Parent topic: Architecture-Specific Functions Reference on page 12

12.7. Interrupt Lock Level: intLock( ) and intUnlock( )

The Intel architecture includes a single interrupt signal for external interrupts, and is able to enable and disable external interrupts to the CPU.

Intel architecture does not have an on-chip interrupt controller, and therefore does not have the capability of controlling the interrupt mask/lock level. The global variable *intLockMask* is set to 1 and is not used by intLock( ). The intLock( ) function simply disables the external interrupt, while the intUnlock( ) function restores the previous state of the signal (that is, enables it if it was previously enabled). Locking the individual external interrupt line or masking the interrupt level is done by a companion interrupt controller device driver such as the **i8259Intr.c** or **ioApicIntr.c**.

The VxWorks SMP kernel does not allow interrupt locking on all the CPUs in the system, and so these functions are not available in VxWorks SMP.

Parent topic: Architecture-Specific Functions Reference on page 12

12.8. Interrupt Lock Level: intCpuLock( ) and intCpuUnlock( )

The Intel architecture includes a single interrupt signal for external interrupts, and is able to enable and disable external interrupts to individual CPU cores.

These functions are called by a VxWorks task or interrupt service routine, and control whether interrupts are enabled or disabled on that context's local CPU.

Intel architecture does not have an on-chip interrupt controller, and therefore does not have the capability of controlling the interrupt mask/lock level. The global variable *intLockMask* is set to 1 and is not used by intCpuLock( ). The intCpuLock( ) function simply disables the external interrupt, while the intCpuUnlock( ) function restores the previous state of the signal (that is, enables it if it was previously enabled). Locking the individual external interrupt line or masking the interrupt level is done by a companion interrupt controller device driver such as the **i8259Intr.c** or **ioApicIntr.c**.

The VxWorks SMP allows interrupt locking on individual local CPUs in the system, and so these functions are available in VxWorks SMP. On the uniprocessor VxWorks, these functions are equivalent to intLock( ) and intUnlock( ).

> Note: On VxWorks SMP, the interrupts on the other CPUs in the system are not affected by these functions. Consequently, they are not suitable as a mutual exclusion mechanism unless all the tasks and/or interrupt service routines participating in the mutual exclusion have a single CPU affinity to the same CPU.

Parent topic: Architecture-Specific Functions Reference on page 12

12.9. IntArchLib: intVecSet2( ) and intVecGet2( )

The functions intVecSet2( ) and intVecGet2( ) replace intVecSet( ) and intVecGet( ), respectively.

intVecSet( ) and intVecGet( ) are kept only for backward compatibility. The functions intVecSet2( ) and intVecGet2( ) include two additional parameters: gate and selector. intVecSet2( ) also includes task gate support. The gate is either **IDT_TRAP_GATE**, **IDT_INT_GATE**, or **IDT_TASK_GATE**; and the selector is either **sysCsExc** or **sysCsInt**.

Parent topic: Architecture-Specific Functions Reference on page 12

12.10. pentiumLib, pentiumALib, and pentiumShow: pentiumXXX( )

Functions that manipulate the memory type range registers (MTRR), performance monitoring counter (PMC), timestamp counter (TSC), machine check architecture (MCA), and model specific registers (MSR) are included.

WNDRVR

The functions are listed in Architecture-Specific Functions Reference on page 12.

Parent topic: Architecture-Specific Functions Reference on page 12

12.11. vxLib, vxALib, and vxShow: vxXXX( )

The vxCpuShow( ) function shows the CPU type, family, model, and supported features.

- The functions vxCr0Get( ), vxCr2Get( ), vxCr3Get( ), and vxCr4Get( ) get the current values from the respective control registers, while the functions vxCr0Set( ), vxCr2Set( ), vxCr3Set( ), and vxCr4Set( ) assign values to the respective control registers.
- The functions vxEflagsGet( ) and vxEflagsSet( ) respectively get and set the **EFLAGS** register.
- The vxDrGet( ) and vxDrSet( ) functions get and set the debug registers. vxDrShow( ) shows the content of the debug registers. These functions are intended to be primitive and generate exceptions if they are not claimed by the debug library.
- The vxTssGet( ) and vxTssSet( ) functions respectively get and set the task register.
- The vxGdtrGet( ), vxIdtrGet( ), and vxLdtrGet( ) functions get the current value of the respective system registers: **GDTR**, **IDTR**, and **LDTR**.
- The function vxLdtrSet( ) sets the content of the local descriptor table register.
- The vxCsGet( ), vxDsGet( ), and vxSsGet( ) functions get the current value of the code segment register, data segment register, and stack segment register, respectively.

Parent topic: Architecture-Specific Functions Reference on page 12

12.12. taskSRSet( )

The taskSRSet( ) function sets its second parameter to the **EFLAGS** register of the specified task.

Parent topic: Architecture-Specific Functions Reference on page 12

13. a.out/ELF-Specific Tools for IA-32

Several tools are specific to the **a.out** format for x86 and Pentium processors, as well as the PC simulator that was used in earlier VxWorks releases.

In the current release, the object module format has been changed to ELF. Therefore, these tools are replaced with **objcopypentium** and no longer supported. For more information, see the reference entries for each tool.

hexDec

Converts an **a.out**-format object file into a Motorola hex record.

aoutToBinDec

Extracts text and data segments from an **a.out** file and writes them to standard output as a simple binary image.

xsymDec

Extracts the symbol table from an **a.out** file.

14. Privilege Protection Operating Mode

Intel architecture supports three operating modes: protected mode, real-address mode, and virtual-8086 mode.

Protected mode is the native operating mode of the 32- and 64-bit processors. All instructions and architectural features are available in this mode for the highest performance and capability. Real-address mode provides the programming environment of the Intel 8086 processor. Virtual-8086 mode lets the processor execute 8086 software in a protected mode, multitasking environment.

For more information about operating modes, see the *Intel® 64 and IA-32 Architectures Software Developer's Manual*.

VxWorks for IA-32 runs in the 32-bit page mode and VxWorks for Intel 64 runs in 64-bit page mode. If real-time processes (RTPs) are not enabled, no privilege protection is used, thus there are no call gates. The privilege level is always 0, which is the most privileged level (supervisor mode). If RTPs are enabled, both level 0 and level 3 (user mode) are used, with the RTP task(s) running at level 3. A call gate is established and used as a system call mechanism to allow RTP task(s) to communicate with the kernel.

VxWorks 64-Bit Address Space Layout on page 34 Review a high-level diagram of VxWorks 64-bit address space layout.

14.1. VxWorks 64-Bit Address Space Layout

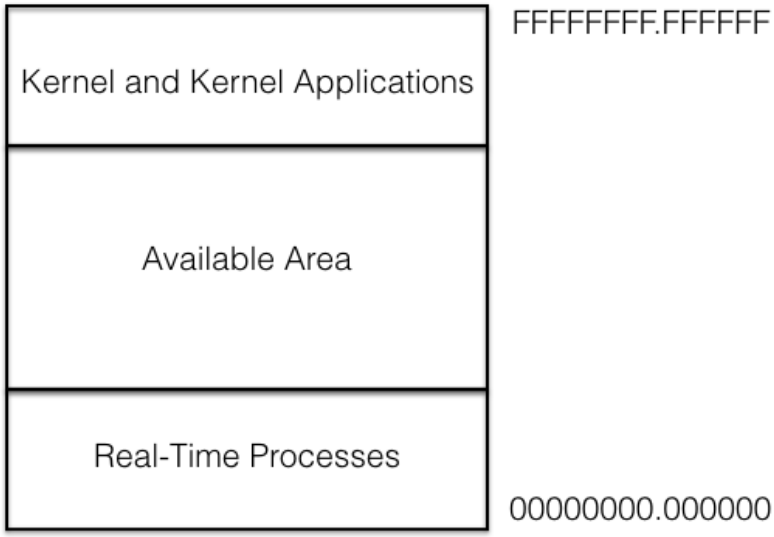
Review a high-level diagram of VxWorks 64-bit address space layout.

The kernel code and kernel application code either loaded in or linked to the kernel are located together on the top, while the real-time processes are located on the bottom. The 64-bit virtual memory space provides a large amount of available space between those two areas. Part of this space is used to hold an additional virtual memory pool used by the kernel and kernel applications. Another part of this space is used to hold shared data regions and shared libraries, as well as a virtual memory pool used by user-mode applications (RTPs).

The implementation detail of this address space layout depends on the processor architecture.

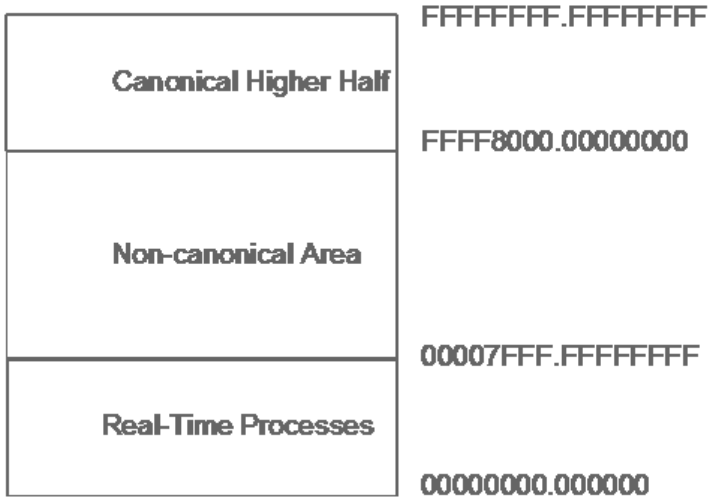
The x86-64 architecture supports only 48 significant bits in its first implementation. At the programming language level, addresses are 64 bits long. The processor requires that, in the first stage, bits 48 through 63 are copies of bit 47 of any address. Otherwise, an exception is raised.

Figure 1. VxWorks 64-bit Address Space



This scheme separates the addresses between a canonical set (abiding by the rule described above) that can be used, and a non-canonical set that cannot be used. The canonical set is divided into a higher half (bits 47 to 63 are all 1's) and a lower half (bits 47 to 63 are all 0's), with the non-canonical set located between them as represented in the next diagram.

Figure 2. Non-canonical Virtual Address Space



The VxWorks address space description and address space management sub-system have to account for this.

The kernel code model is used for the kernel environment (that is, the OS code and kernel applications) while the small code model is the default for the RTP environment.

For information about X86-64 ABI and descriptions of the code models, see the *System V Application Binary Interface AMD64 Architecture Processor Supplement*.

Kernel Code Model

The kernel of an operating system is usually rather small but runs in the negative half of the address space. So we define all symbols to be in the range from 264 - 231 to 264 - 224 or from 0xffffffff80000000 to 0xffffffffff000000.

This code model has advantages similar to those of the small model, but allows encoding of zero extended symbolic references only for offsets from 231 to 231 + 224 or from 0x80000000 to 0x81000000. The range offsets for sign extended reference changes to 0 to 231 + 224 or 0x00000000 to 0x81000000.

Small Code Model

The virtual address of code executed is known at link time. Additionally all symbols are known to be located in the virtual addresses in the range from 0 to 231 - 224 - 1 or from 0x00000000 to 0x7effffff.

This allows the compiler to encode symbolic references with offsets in the range from - (231) to 224 or from 0x80000000 to 0x01000000 directly in the sign extended immediate operands, with offsets in the range from 0 to 231 - 224 or from 0x00000000 to 0x7f000000 in the zero extended immediate operands and use instruction pointer relative addressing for the symbols with offsets in the range - (224) to 224 or 0xff000000 to 0x01000000.

This is the fastest code model and we expect it to be suitable for the vast majority of programs.

Large Code Model

The large code model makes no assumptions about addresses and sizes of sections.

The compiler is required to use the **movabs** instruction, as in the medium code model, even for dealing with addresses inside the text section. Additionally, indirect branches are needed when branching to addresses whose offset from the current instruction pointer is unknown.

It is possible to avoid the limitation on the text section in the small and medium models by breaking up the program into multiple shared libraries, so this model is strictly only required if the text of a single function becomes larger than what the medium model allows.

Parent topic: Privilege Protection Operating Mode on page 34

15. Byte Order

The IA-32 and Intel 64 byte order is little-endian, but network applications must convert some data to a standard network order, which is big-endian.

In particular, in network applications, be sure to convert the port number to network byte order using htons( ).

16. Caches

The CD and NW flags in CR0 control the overall caching of system memory. The PCD and PWT flags in CR3 control the caching of the page directory.

The **PCD** and **PWT** flags in the page directory or page table entry control page-level caching. In **cacheLib**, the **WBINVD** instruction is used to flush the cache if the **CLFLUSH** instruction is not supported by the processor.

P7 (Pentium 4), Atom, Core, and Nehalem family processors include an L1 data cache and an L2 unified cache. The **CLFLUSH** instruction allows the selected cache line to be flushed from memory.

> Note: Processors based on the NetBurst architecture also include a trace cache that caches decoded instructions. Atom, Core, and Nehalem processors do not have trace caches.

The 64-bit processors include separate L1 instruction and data caches, and a unified internal L2 cache. The processor MESI data cache protocol maintains consistency with internal L1 and L2 caches, caches of other processors, and with an external cache in both update policies. The operation of the MESI protocol is transparent to software.

17. FPU, MMX, SSE/2/3, SSSE3, SSE4, and AVX/AVX2 Support

The x87 math coprocessor and on-chip floating-point instructions are software compatible, and are supported by VxWorks using the INCLUDE_HW_FP configuration macro.

There are multiple types of floating-point contexts:

- The first type is 108 bytes and is used for older FPUs (i80387, i80487, Pentium) and older MMX technology.
- The second type is 512 bytes and is used for newer FPUs, newer MMX technology, and SSE technology (Pentium 4, Atom, Core, and Nehalem). For SSE instruction generation, use the corresponding compiler switch when creating RTPs. The following table provides a list switches.

Table 1. Compiler Switches

Instruction	Switch
SSE	**-msse**
SSE2	**-msse2**
SSE3	**-msse3**
SSSE3	**-mssse3**
SSE4.1	**-msse4.1**
SSE4.2	**-msse4.2**

- The third type is 1024 bytes and is used for AVX technology (second generation Core) and AVX2 (for Haswell boards and later). For AVX instruction generation, you must use the **-mavx** switch for the GNU compiler when creating an RTP, DKM, or VIP.

The type of floating-point context is determined by the coprocessor abstraction of the floating-point in use, and VxWorks automatically uses the appropriate functions to interface to the internal functions.

Saving and restoring floating-point registers adds to the context switch time of a task. Therefore, floating-point registers are not saved and restored for every task. Only those tasks spawned with the task option **VX_FP_TASK** will have floating-point state, **MMX** technology state, and streaming **SIMD** state saved and restored. If a task executes any floating-point operations, **MMX** operations, or streaming **SIMD** operations, it must be spawned with **VX_FP_TASK**.

For additional information about **VX_FP_TASK**, see the *VxWorks 7 Application Programmer's Guide: Task Options*.

Executing floating-point operations from a task spawned without the **VX_FP_TASK** option results in an undefined instruction exception or serious and difficult to find errors. VxWorks includes a special API and mechanism to detect this type of illegal, unintentional, or accidental floating-point operation. To use this mechanism, call fppArchSwitchHook( ) to connect the new task switch hook function. Then call the enabler, fppArchSwitchHookEnable( ), with a parameter TRUE (1). The mechanism is disabled using the FALSE (0) parameter.

This mechanism is disabled in the default VxWorks configuration.

There are six floating-point context exceptions that can send an exception to the CPU. They are controlled by the exception mask bits of the control word register. VxWorks disables these exceptions in the default configuration. The exceptions are: precision, overflow, underflow, division by zero, denormalized operand, and invalid operation.

For more information on instruction mixing, see:

http://www.intel.com/

18. Segmentation

In the default configuration—that is, error detection and reporting and RTPs disabled—three code segments and one data segment are defined in the global descriptor table (GDT).

The GDT is defined as table sysGdt[ ] in **sysALib.s**, and is copied to the destination address at (**LOCAL_MEM_LOCAL_ADRS** + **GDT_BASE_OFFSET**). The defined code and data segments are:

- supervisor code/data segment with privilege level 0 (PL0)

WNDRVR

- interrupt/exception code segment with privilege level 0 (PL0)

They are fully overlapped in the 4 GB, 32-bit address space (flat model). These segments are used when the execution mode of a task is changed during its lifetime.

When RTPs are enabled, an additional three segments, a call gate, and a TSS descriptor are added to the GDT. The three segments are level 3 (PL3) for use by user-mode RTP tasks. The segments include one data, one code, and one stack segment.

IA-32

The call gate and **TSS** descriptor are used by the system call mechanism to allow a mode switch to occur when a system call is made.

When error detection and reporting is enabled, the **IDT** gets a task gate entry for page fault management. The GDT gets two **TSS** entries (one for **OSM** save information and one for **OSM** restore information) and one task gate entry. An **LDT** entry is also established for context switching through **TSS**.

Intel 64 (and IA-32 Supported by the itl_generic BSP)

Because VxWorks 64-bit will always have the MMU functionality available, real-time processes (RTPs) are supported only in their overlapped virtual memory mode; this mode brings better application start-up time performances. Therefore, the relocatable user-mode executable functionality is not supported on 64-bit VxWorks.

For more information about overlapped virtual memory, see the *VxWorks 7 Platform Programmer's Guide: Kernel Virtual Memory Management*.

Notes for RTPs in VxWorks 64-bit platforms:

- Static RTP 64-bit executables are always built as absolutely linked executables (with no relocation support) and dynamic 64-bit executables have dynamic relocation information.
- VxWorks RTP executable images used on a 64-bit VxWorks platform (64-bit **.vxe** files) are fully linked executables.
- A 64-bit **.vxe** file must be an ELF object,**EI_CLASS**must be set to **ELFCLASS64**, and the object file type must be **ET_EXEC**.
- Do not build the 64-bit **.vxe** files with relocation information. 32-bit VxWorks relocates RTP executables upon loading, and thus requires relocation information to be present in the executable image.
- Dynamically linked 64-bit **.vxe** files can carry dynamic relocation information.

  A 64-bit **.vxe** file must contain two program headers of type PT_LOAD, one containing executable text and one containing data.

  The loadable segment containing executable text (the text segment), must be at the lower address; the **p_vaddr** of the text segment must be 0x200000. In other words, a 64-bit **.vxe** file must be built at load address of 0x200000, the page size is 0x1000 bytes. (For IA-32, the load address is 0x400000.)

  The other loadable segment contains program data. This segment begins on a page following the end of the text segment. In other words, the size of the text segment is rounded up the next page boundary. Consequently, **p_align** may be set to either 0 or 1.

  Dynamically linked 64-bit **.vxe** files cannot contain a **PT_INTERP** type header (VxWorks ignores this program header.).

  64-bit **.vxe** files built for the x86-64 architecture can be built using the small, medium, or large code models. They are built for the small code model by default.

When error detection and reporting is enabled, the IDT gets a call gate entry for page fault management.

19. Paging with MMU

When paging is used, the linear address space is divided into fixed-size pages (4 KB is the default configuration).

Note: This information does not apply to the **itl_generic** BSP which uses the newer VxWorks 7 memory model. For more information, see the memory management information in the *VxWorks 7 Platform Programmer's Guide: VxWorks 7 Memory Model*.

Entries in the page directory point to page tables and entries in the page table point to pages in physical memory. Bits 22 through 31 of the linear address space provide an offset to an entry in the page directory. Bits 12 through 21 of the linear address space provide an offset to an entry in the selected page table. Bits 0 through 11 provide an offset to a physical address in the page.

If **INCLUDE_MMU_BASIC** component is enabled, VxWorks enables the MMU with the **sysPhysMemDesc[ ]** table which includes PCI memory mapping information. This is the default VxWorks configuration.

If you have other memory-mapped devices, and if **INCLUDE_MMU_BASIC** is included (the default), you may need to add your device address space into the MMU table by manually editing the MMU configuration structure **sysPhysMemDesc[ ]** in **sysLib.c**. For information on editing this structure, see the *VxWorks 7 Platform Programmer's Guide*. Do not overlap any existing MMU entries and be sure all entries are page aligned. Wind River recommends that you also maintain a 1:1 correlation between virtual and physical memory because VxWorks and all tasks use a common address space. Attempts to access areas not mapped as valid in the MMU result in page faults.

Note: Paging is required for VxWorks SMP.

P7 (Pentium 4), Atom, Core, Nehalem, Sandybridge, and Haswell MMU on page 39 The enhanced MMU on P7, Atom, Core, Nehalem, Sandybridge, and Haswell family processors supports two additional page attribute bits.
Global Descriptor Table (GDT) on page 40 The GDT is defined as the table **sysGdt[ ]** in **sysALib.s**.

19.1. P7 (Pentium 4), Atom, Core, Nehalem, Sandybridge, and Haswell MMU

The enhanced MMU on P7, Atom, Core, Nehalem, Sandybridge, and Haswell family processors supports two additional page attribute bits.

The global bit (G) indicates a global page when set. When a page is marked global, and the page global enable (PGE) bit in register CR4 is set, the page-table or page-directory entry for the page is not invalidated in the TLB when register CR3 is loaded. This bit is provided to prevent frequently used pages (such as pages that contain kernel or other operating system or executive code) from being flushed from the TLB.

The page-level write-through/write-back bit (PWT) controls the write-through or write- back caching policy of individual pages or page tables. When the PWT bit is set, write-through caching is enabled for the associated page or page table. When the bit is clear, write-back caching is enabled for the associated page and page table.

The following macros describe these attribute bits in the physical memory descriptor table **sysPhysMemDesc[ ]** in **sysLib.c**:

MMU_ATTR_CACHE_COPYBACK (or **VM_STATE_WBACK**)

Use write-back cache policy for the page.

MMU_ATTR_CACHE_OFF (or **VM_STATE_CACHEABLE_NOT**)

Disable caching for the page.

VM_STATE_GLOBAL

Set page global bit.

VM_STATE_GLOBAL_NOT

Do not set page global bit.

Support is provided for 4 KB pages. The linear address for 4 KB pages is divided into three sections. These sections are as follows:

- Page directory entry – bits 22 through 31
- Page table entry – bits 12 through 21
- Page offset – bits 0 through 11

Parent topic: Paging with MMU on page 38

19.2. Global Descriptor Table (GDT)

The GDT is defined as the table **sysGdt[ ]** in **sysALib.s**.

The table begins with five entries as follows:

- a null entry
- an entry for program code
- an entry for program data
- an entry for exceptions
- an entry for interrupts

If error detection and reporting is enabled, an additional entry is added for task gate management of the Operating System Miscue (OSM) stack as well as two TSS entries (one for OSM save information and one for OSM restore information). If RTPs are enabled, an entry is provided for level 3 (user-mode) support. The table is initially set to have an available memory range of 0x0-0xffffffff. For boards that support PCI, the component **INCLUDE_PCI_BUS** must be included in the VIP, and VxWorks will not alter the pre-set memory range. This memory range is available at run-time with the MMU configuration.

If **INCLUDE_PCI_BUS** is not included in the VIP (the default for boards that do not support PCI), VxWorks adjusts the GDT using the sysMemTop( ) function to check the actual memory size during system initialization and set the table to have an available memory range of 0x0-sysMemTop( ). This causes a general protection fault to be generated for any memory access outside the memory range 0x0-sysMemTop( )

Parent topic: Paging with MMU on page 38

20. Page Size Optimization

A dynamic method for mapping addresses as large pages is available through the VxWorks **vmLib** function, vmPageOptimize( ) (currently, only the **mmuPro32Lib** library supports this option which includes Pentium 4 and later processors).

> Note: This information does not apply to the **itl_generic** BSP which uses the newer VxWorks 7 memory model. For more information, see the memory management information in the *VxWorks 7 Platform Programmer's Guide: VxWorks 7 Memory Model*.

Page size optimization support for MMUs allows more efficient management of virtual memory with fewer translation table walks or translation lookaside buffer (TLB) misses. The vmPageOptimize( ) function allows default sized 4 KB MMU pages to be coalesced into 4 MB pages for contiguous memory blocks having the same attributes. De-optimization is performed automatically when necessary. For example, if the attributes are changed for part of a memory block that is mapped to a 4 MB MMU page, it is broken up into 4 KB pages and the new attributes are applied to the requested pages only.

> Note: MMU page sizes other than 4 KB and 4 MB are not supported for this architecture.

Intel architecture support for vmPageOptimize( ) includes the following limitation that is not described in the reference entry for the vmPageOptimize( ) function:

Note: Once a page optimization operation (for example, vmPageOptimize( )) is performed, attempts to change page attributes during an interrupt are limited. Calls to vmStateSet( ) during interrupt processing fail if the Virtual Memory Management special purpose attribute **MMU_ATTR_NO_BLOCK** is set for the range of addresses. The call to vmStateSet( ) returns **S_mmuLib_ISR_CALL_BLOCKED** in this case. **MMU_ATTR_NO_BLOCK** is a per-page attribute and cannot be set globally.

Optimization of the entire kernel memory space can be done automatically at start-up by adding the **INCLUDE_PAGE_SIZE_OPTIMIZATION** component to a VxWorks project. (The **INCLUDE_MMU_OPTIMIZE** component is also required, but is automatically included by the project facility.) Some blocks of memory used by VxWorks, such as those allocated for MMU page tables and for error detection and reporting, may prevent optimization of certain areas. This is because those blocks of memory may have different attributes than the rest of memory in the same 4 MB-aligned address range.

21. Ring-Level Protection

The processor's segment protection mechanism recognizes four privilege levels numbered 0 to 3. The greater numbers have fewer privileges.

VxWorks uses privilege level 0 (PL0) when executing kernel, exceptions, and interrupt code. Privilege level 3 (PL3) is used when executing RTP task code.

22. Interrupts

Interrupt service routines (ISRs) are executed in supervisor mode (PL0) with the dedicated interrupt stack or the task's supervisor stack.

The dedicated interrupt stack is the default, and requires software manipulation. Conversely, the task supervisor stack does not require the OS to perform any software intervention. You can control the trade-off between performance and memory consumption by selecting the stack used with an ISR. If you want faster interrupt response time, use the task stack; if you want to save on memory consumption, use the dedicated interrupt stack.

To disable the dedicated interrupt stack, perform

```
intStackEnable(FALSE)
```

in the task level. The default size of the interrupt stack is defined by VxWorks image project (VIP) component parameter **ISR_STACK_SIZE**. It is set to 0x3E8 bytes (1000 in decimal) in 32-bit mode and in 64-bit mode it is set to 0x2000 bytes.

Interrupt Handling on page 41 Exceptions and the NMI interrupt are assigned vectors in the range of 0 through 31.
Interrupt Descriptor Table on page 42 The interrupt descriptor table (IDT) occupies the address range from LOCAL_MEM_LOCAL_ADRS + 0x0 to 0x800
Operating System Miscue (OSM) Stack on page 43 The OSM stack is needed for handling and recovery of stack overflow/underflow conditions and is triggered immediately following a page fault (stack overflow/underflow conditions are seen as a page fault).
BOI and EOI on page 43 The interrupt handler calls intEnt( ) and saves the volatile registers (eax, edx, and ecx).
Interrupt Mode on page 43 VxWorks for Intel architecture only supports symmetric I/O interrupt mode.

22.1. Interrupt Handling

Exceptions and the NMI interrupt are assigned vectors in the range of 0 through 31.

Unassigned vectors in this range are reserved for possible future use. The vectors in the range 32 to 255 are provided for maskable interrupts.

The Intel architecture enables or disables all maskable interrupts with the IF flag in the EFLAGS register. An external interrupt controller handles multi-level priority interrupts. One popular interrupt controller is the Intel 8259 PIC (programmable interrupt controller) which is supported by VxWorks as an interrupt controller driver.

The Fully Nested Mode and the Special Fully Nested Mode are supported and configurable in the BSP. In the Special Fully Nested Mode, when an interrupt request from a slave PIC is in service, the slave is not locked out from the master 's priority logic and further interrupt requests from higher-priority IRQs within the slave are recognized by the master and initiate interrupts to the processor.

The PIC (8259A) IRQ0 is hard-wired to the PIT (8253) channel 0 in a PC motherboard. IRQ0 is the highest priority in the 8259A interrupt controller. Thus, the system clock interrupt handler blocks all lower-level interrupts. This may cause a delay of the lower-level interrupts in some situations even though the system clock interrupt handler finishes its job without any delay. This is quite natural from the hardware point of view, but may not be ideal from the application software standpoint. The following modes are supplied to mitigate this situation by providing the corresponding configuration macros in the BSP. The three mutually exclusive modes are Early EOI Issue in IRQ0 ISR, Special Mask Mode in IRQ0 ISR, and Automatic EOI Mode. For more information, see your BSP documentation.

The intLock( ) and intUnlock( ) functions control the IF flag in the **EFLAGS** register. The sysIntEnablePIC( ) and sysIntDisablePIC( ) functions control a specified PIC interrupt level.

Note: PIC mode is not supported by the **itl_generic BSP**.

Parent topic: Interrupts on page 41

22.2. Interrupt Descriptor Table

The interrupt descriptor table (IDT) occupies the address range from LOCAL_MEM_LOCAL_ADRS + 0x0 to 0x800

Vector numbers 0x0 to 0x1f are handled by the default exception handler. Vector numbers 0x20 to 0xff are handled by the default interrupt handler.

The call gate (trap gate for IA-32) is used for exceptions (vector numbers 0x0 - 0x1f). The configurable global variable sysIntIdtType, which can be set to either call gate (trap gate for IA-32) or interrupt gate in the BSP, is used for interrupts (vector numbers 0x20 - 0xff). For Intel 64, the default is to use the interrupt gate.

The difference between an interrupt gate and a call gate (trap gate for IA-32) is its effect on the IF flag: using an interrupt gate clears the IF flag, which prevents other interrupts from interfering with the current interrupt handler.

Each vector entry in the IDT contains the following information:

- offset (offset to the interrupt handler)
- selectors (sysCsExc(0x0018), fourth descriptor (code) in GDT for exceptions; or sysCsInt(0x0020), fifth descriptor (code) in GDT for interrupts)
- descriptor privilege level (3)
- descriptor present bit (1)

Note: If you are migrating from 32-bit VxWorks 64-bit VxWorks, you must use an interrupt gate. Trap gates are not supported for Intel 64.

Parent topic: Interrupts on page 41

WNDRVR

22.3. Operating System Miscue (OSM) Stack

The OSM stack is needed for handling and recovery of stack overflow/underflow conditions and is triggered immediately following a page fault (stack overflow/underflow conditions are seen as a page fault).

Issues that exist when possible stack overflow/underflow occurs are passed to the OSM stack. A call gate (task gate for IA-32) is used for the page fault. This allows VxWorks to jump to the OSM task function. The task function then establishes an OSM task, reconfigures both OSM TSS entries and the segment descriptors to their proper states before the exception occurs, and then enters the exception handler as if handling a standard page fault. By using a new "safe" stack, the OSM allows the user to debug the issue that caused the stack problem.

Parent topic: Interrupts on page 41

22.4. BOI and EOI

The interrupt handler calls intEnt( ) and saves the volatile registers (eax, edx, and ecx).

It then calls the ISR, which is usually written in C. Finally, the handler restores the saved registers and calls intExit( ). The beginning-of-interrupt (BOI) and end-of-interrupt (EOI) functions are called before and after the ISR. The BOI function ascertains whether or not the interrupt is stray; if it is stray, the BOI function jumps to intExit( ). If the interrupt is not stray, the BOI function returns to the caller. The EOI function issues an EOI signal to the interrupt controller, if necessary.

Some device drivers (depending on the manufacturer, the configuration, and so on) generate a stray interrupt on IRQ7 (which is used by the parallel driver), and on IRQ15. The global variable sysStrayIntCount is incremented each time such an interrupt occurs, and a dummy ISR is connected to handle these interrupts. For more information about sysStrayIntCount, see your BSP documentation.

Parent topic: Interrupts on page 41

22.5. Interrupt Mode

VxWorks for Intel architecture only supports symmetric I/O interrupt mode.

The **itl_generic** BSP supports symmetric I/O interrupt mode, which uses both the local APIC and I/O APIC. Both UP and SMP kernel configurations are built with symmetric I/O interrupt mode enabled. Neither virtual wire or PIC mode is supported by this VxWorks release.

For more information, see *itl_generic BSP Supplement Guide: Interrupts*.

Parent topic: Interrupts on page 41

23. System Management Mode

VxWorks does not support System Management Mode (SMM) for IA-32 processors.

In accordance with this, VxWorks does not cause the generation of System Management Interrupts (SMI) through the advanced programmable interrupt controller (APIC). However, some hardware targets that are supported by VxWorks generate SMI through the SMM interrupt pin (SMI#) under certain conditions. These SMI have the potential to impact real-time performance of the system due to the unexpected and unmaskable overhead associated with servicing the SMI. You must ensure that the overhead associated with SMI (if any) is accounted for in any real-time system analysis.

For more information on SMM and how it is implemented on your system, see your target hardware documentation.

WNDRVR

24. Exceptions

Exception handlers are executed in supervisor mode (PL0) with the task supervisor stack. All exceptions switch to the exception stack.

Exceptions differ from interrupts, with regard to the operating system, because interrupts are executed at the interrupt level and exceptions are executed at the task level.

After saving all registers on the supervisor stack, the task prints out the exception messages and then suspends itself. Execution can be resumed with the information stored in the supervisor stack. The processor generates an exception stack frame in one of two formats, depending on the exception type. The types are as follows:

(EIP + CS + EFLAGS) or (ERROR + EIP + CS + EFLAGS)

The CS (Code Selector) register is taken from the vector table entry. That entry is the *sysCsExc* global variable defined in the BSP.

25. Stack Management

The task stack is used for task-level execution.

The intEnt( ) and intExit( ) functions are used to switch to and from the interrupt stack, if the interrupt stack is enabled. The default size of the interrupt stack is defined by VxWorks image project (VIP) component parameter **ISR_STACK_SIZE**. It is set to 0x3E8 bytes (1000 in decimal) in 32-bit mode and in 64-bit mode it is set to 0x2000 bytes.

26. Context Switching

Context switching is handled in software by the VxWorks kernel.

For IA-32, hardware multitasking through task gates and TSS descriptors is not used for normal context switching. The switch is accomplished by building a dummy exception stack frame and then using the IRET instruction to make the contents of the stack frame the new processor state.

27. Machine Check Architecture (MCA)

The machine check exception (interrupt -18) is used to signal hardware-related errors, such as a parity error on a read cycle, generate a machine check exception, record information about the machine check errors, and provide the basis for extended error logging capability.

MCA is enabled by default and its status registers are set to zero in pentiumMcaEnable( ) in sysHwInit( ). These registers are accessed by pentiumMsrSet( ) and pentiumMsrGet( ).

28. Registers

28.1. Memory Type Range Register (MTRR)

MTRRs are a feature of the Intel Architecture that allow the processor to optimize memory operations for different types of memory, such as RAM, ROM, frame buffer memory, and memory-mapped I/O. MTRRs configure an internal map of how physical address ranges are mapped to various types of memory.

WNDRVR

The processor uses this internal map to determine the cache ability of various physical memory locations and the optimal method of accessing memory locations.

For example, if a memory location is specified in an MTRR as write-through memory, the processor handles accesses to this location either by reading data from that location in lines and caching the read data or by mapping all writes to that location to the bus and updating the cache to maintain cache coherency. In mapping the physical address space with MTRRs, the processor recognizes five types of memory: uncacheable (UC), write-combining (WC), write-through (WT), write-protected (WP), and write-back (WB).

The MTRR table is defined as follows:

```
typedef struct mtrr_fix /* MTRR - fixed range register */
{
char type[8]; /* address range: [0]=0-7 ... [7]=56-63 */
} MTRR_FIX;
typedef struct mtrr_var /* MTRR - variable range register */
{
long long int base; /* base register */
long long int mask; /* mask register */
} MTRR_VAR;
typedef struct mtrr /* MTRR */
{
int cap[2]; /* MTRR cap register */
int deftype[2]; /* MTRR defType register */
MTRR_FIX fix[11]; /* MTRR fixed range registers */
MTRR_VAR var[256]; /* MTRR variable range registers */
} MTRR;
```

28.2. Model-Specific Register (MSR)

Model specific registers (MSRs) are used to control hardware functions in the processor or to monitor processor activity.

The new registers control the debug extensions, the performance counters, the machine-check exception capability, the machine check architecture, and the MTRRs. The MSRs can be read from and written to using the **RDMSR** and **WRMSR** instructions, respectively.

29. Counters

Performance Monitoring Counters (PMCs)

Intel Architecture processors have two performance-monitoring counters for use in monitoring internal hardware operations.

These counters are duration or event counters that can be programmed to count any of approximately 100 different types of events, such as the number of instructions decoded, number of interrupts received, or number of cache loads.

PMCs are initialized in sysHwInit( ).

Timestamp Counter (TSC)

Intel Architecture provides a 64-bit timestamp counter that is incremented every processor clock cycle.

The counter is incremented even when the processor is halted by the HLT instruction or the external STPCLK# pin. The timestamp counter is set to 0 following a hardware reset of the processor. The RDTSC instruction reads the timestamp counter and is guaranteed to return a monotonically increasing unique value whenever executed, except for 64-bit counter wraparound. Intel

guarantees, architecturally, that the timestamp counter frequency and configuration is such that it will not wraparound within 10 years after being reset to 0. The period for counter wrap is several thousands of years in these processors.

30. Advanced Programmable Interrupt Controller

30.1. Local APIC/xAPIC

The local APIC/xAPIC module is a driver for the local advanced programmable interrupt controller in Intel Architecture families.

The local APIC/xAPIC is included in select Pentium 4 processors and later and select Intel 64-based processors. On these processors, the presence or absence of an on-chip local APIC can be detected using the **CPUID** instruction. When the **CPUID** instruction is executed, bit 9 of the feature flags returned in the EDX register indicates the presence (set) or absence (clear) of an on-chip local APIC.

The local APIC performs two main functions for the processor:

- It processes local external interrupts that the processor receives at its interrupt pins as well as local internal interrupts generated by software.
- In multiple-processor systems, it communicates with an external I/O APIC chip. The external I/O APIC receives external interrupt events from the system as well as interprocessor interrupts from the processors on the system bus and distributes them to the processors on the system bus. The I/O APIC is part of Intel's system chip set.

The local APIC controls the dispatching of interrupts (to its associated processor) that it receives either locally or from the I/O APIC. It provides facilities for queuing, nesting, and masking interrupts. The local APIC handles the interrupt delivery protocol with its local processors as well as accesses to APIC registers. In addition, it manages interprocessor interrupts and remote APIC register reads. A timer on the local APIC allows local generation of interrupts, and local interrupt pins permit local reception of processor-specific interrupts.

The local APIC (called the xAPIC) can be disabled and used in conjunction with a standard 8259A-style interrupt controller. Disabling the local APIC can be done in software for Intel processors.

The local xAPIC and I/O xAPIC communicate with one another through the processor ’s system bus.

The base address of the local APIC and I/O APIC is taken from the multiprocessor (MP) configuration table (for more information, see Intel MP Specification Version 1.4) or the **IA32_APIC_BASE MSR**. If the local APIC driver is unable to find the addresses, it uses **LOAPIC_BASE** and **IOAPIC_BASE** as defined in the BSP. This driver contains three functions for use. The functions are:

vxbLoApicIntrInit( )

Initializes the local APIC for the chosen interrupt mode. Do not call this function directly, it is only for use by the VxWorks VxBus system code. For more information on VxBus, refer to the *VxWorks 7 BSP and Driver Guide*.

loApicIntrShow( )

Shows the local APIC registers.

mpBiosShow( )

Shows the MP configuration table.

30.2. Local x2APIC

The local x2APIC is the most recent generation of the Intel's programmable interrupt controller, introduced with the Nehalem microarchitecture.

The major improvements of the x2APIC address the number of supported CPUs and performance of the interface.

WNDRVR

The x2APIC now uses 32 bits to address CPUs, allowing to address up to 2^32 – 1 CPUs using the physical destination mode. The logical destination mode now works differently and introduces clusters. Using this mode, the user can address up to 2^20 – 16 CPUs.

The x2APIC architecture also provides backward compatibility modes to the original Intel APIC Architecture (introduced with the Pentium/P6) and with the xAPIC architecture (introduced with the Pentium 4).

Some customer reference boards such as Denverton exposes the option to the user to decide if the x2APIC is enabled or not. By default the x2APIC is enabled, which means the BIOS will leave the x2APIC enabled to the operating system.

The user can configure the mode of local APIC using the APIC_MODE_PARAM parameter, which means that the user can override the mode of local APIC that the operating system inherits from the BIOS.

The parameter can be set using the wrtool command-line utility, for example:

- ```
  prj vip parameter set APIC_MODE_PARAM 0
  ```

  The parameter value of 0 is for inheriting BIOS's setting.

- ```
  prj vip parameter set APIC_MODE_PARAM 1
  ```

  The parameter value of 1 is for disabling x2APIC.

- ```
  prj vip parameter set APIC_MODE_PARAM 2
  ```

  The parameter value of 2 is for enabling x2APIC.

Note: The default value of APIC_MODE_PARAM is 0, which means by default VxWorks will inherit the BIOS's setting of the mode of local APIC.

30.3. APIC Use with Interrupt Modes

While the MP specification defines three interrupt modes: virtual wire mode, symmetric I/O mode, and PIC mode, VxWorks only supports one.

VxWorks supports symmetric I/O mode only and sets this as the VxWorks interrupt mode by default.

In symmetric I/O mode, the local and I/O APICs are fully functional, and interrupts are generated and delivered to the processors by the APICs. Any interrupt can be delivered to any processor. This is the only interrupt mode supported for uniprocessor and multiprocessor VxWorks systems.

30.4. Interrupt Priority

The local and I/O APICs support interrupts in the range of 32 to 255. Interrupt priority is implied by its vector, according to the following relationship: priority = vector / 16.

Here the quotient is rounded down to the nearest integer value to determine the priority, with 1 being the lowest and 15 the highest. Because vectors 0 through 31 are reserved for exclusive use by the processor, the priority of user defined interrupts range from 2 to 15. A value of 15 in the interrupt class field of the task priority register (TPR) masks off all interrupts that require interrupt service. To avoid losing interrupts, software should allocate no more than 2 interrupt vectors per priority. P7 (Pentium 4) family processors expand this support by allowing two interrupts per vector rather than per priority level.

The vector range of MSI (Message Signaled Interrupts) is from LOAPIC_VECTOR_MSI_NUM_START (default value of 0xB0) to LOAPIC_VECTOR_MSI_NUM_END (default value of 0xDF).

The parameters can be set using the wrtool command-line utility, for example:

```
prj vip parameter set LOAPIC_VECTOR_MSI_NUM_START 64
```
```

```
prj vip parameter set LOAPIC_VECTOR_MSI_NUM_END 111
```

The vector range of PCI (Programmable Interrupt Controller) interrupts starts from IOAPIC_VECTOR_INT_NUM_PCI (default value of 0x3F). There are a total of 8 PCI vectors that continuously decrease by a value of 1 from IOAPIC_VECTOR_INT_NUM_PCI.

The parameter can be set using the wrtool command-line utility, for example:

```
prj vip parameter set IOAPIC_VECTOR_INT_NUM_PCI 120
```

The vector range of ISA (Industry Standard Architecture) interrupts starts from IOAPIC_VECTOR_INT_NUM_ISA (default value of 0xAC). There are a total of 16 ISA vectors that continuously decrease by a value of 4 from IOAPIC_VECTOR_INT_NUM_ISA.

The parameter can be set using the wrtool command-line utility, for example:

```
prj vip parameter set IOAPIC_VECTOR_INT_NUM_ISA 80
```

Note: If the user changes the default value of IOAPIC_VECTOR_INT_NUM_ISA, then the following vectors decreases by a value of 1 instead of 4.

The vector range of MISC interrupts is from IOAPIC_VECTOR_INT_NUM_MISC_START (default value of 0xAF) to IOAPIC_VECTOR_INT_NUM_MISC_END (default value of 0x30).

The parameters can be set using the wrtool command-line utility, for example:

```
prj vip parameter set IOAPIC_VECTOR_INT_NUM_MISC_START 60
```

```
prj vip parameter set IOAPIC_VECTOR_INT_NUM_MISC_END 40
```

Note: The kernel will fail to start if the user changes the MSI, PCI, ISA or MISC interrupt vector ranges without adhering to the following conditions:

- The range of MSI, PCI, ISA and MISC must not overlap.
- The range of MSI must be at least 48 vectors.
- The range of PCI must be 8 vectors.
- The range of ISA must be 16 vectors.

30.5. I/O APIC/xAPIC

The I/O APIC/xAPIC module is a driver for the I/O advanced programmable interrupt controller for Intel processors.

The I/O APIC/xAPIC is included in some Intel system chip sets, such as ICH2. Software intervention may be required to enable the I/O APIC/xAPIC on some chip sets.

The 8259A interrupt controller is intended for use in uniprocessor systems; I/O APIC can be used in either uniprocessor or multiprocessor systems. The I/O APIC handles interrupts differently than the 8259A. Briefly, these differences are:

- Method of Interrupt Transmission. The I/O APIC transmits interrupts through a 3-wire bus and interrupts are handled without the need for the processor to run an interrupt acknowledge cycle.
- Interrupt Priority. The priority of interrupts in the I/O APIC is independent of the interrupt number. For example, interrupt 10 can be given a higher priority than interrupt 3.
- More Interrupts. The I/O APIC design can theoretically support up to 256 interrupts.

The I/O APIC unit consists of a set of interrupt input signals, a 256-entry by 64-bit interrupt redirection table, programmable registers, and a message unit for sending and receiving APIC messages over the APIC bus or the front-side (system) bus.

Note: The convention is for I/O APIC implementations to be limited to 24 interrupts as this was the number of external pins available on the original **Intel 82093AA** standalone I/O APIC chip implementation. However, systems exist in the field with more than 24 inputs on a single I/O APIC. Consequently, the VxWorks programming interface is implemented to allow for up to 256 redirection table entries.

I/O devices inject interrupts into the system using one of the I/O APIC interrupt lines. The I/O APIC selects the corresponding entry in the redirection table and uses the information in that entry to format an interrupt request message. Each entry in the redirection table can be individually programmed to indicate edge/level sensitive interrupt signals, the interrupt vector and priority, the destination processor, and how the processor is selected (statically and dynamically). The information in the table is used to transmit a message to other APIC units (through the APIC bus or the front-side (system) bus).

I/O APIC is used in the symmetric I/O mode. The base address of the I/O APIC is determined in ioApicInit( ) and stored in the global variables **ioApicBase** and **ioApicData**. The ioApicInit( ) function initializes the I/O APIC with information stored in **ioApicRed0_15** and **ioApicRed16_23**. **ioApicRed0_15** is the default lower 32-bit value of the redirection table entries for IRQ 0 to IRQ 15 which are edge triggered positive high, **ioApicRed16_23** is the default value for IRQ 16 to IRQ 23 which are level triggered positive low. The ioApicRedSet( ) and ioApicRedGet( ) functions are used to access the redirection table. The ioApicEnable( ) function enables the I/O APIC or xAPIC. The ioApicIrqSet( ) function sets the specific IRQ to be delivered to the specific local APIC. The ioApicShow( ) function shows the I/O APIC registers. This implementation supports multiple I/O APIC instances.

30.6. Local APIC Timer

The local APIC timer library contains functions for the timer in the Intel local APIC/xAPIC in P7 (Pentium 4), Atom, Core, Nehalem, and Intel 64-based processor families.

The local APIC contains a 32-bit programmable timer for use by the local processor. This timer is configured through the timer register in the local vector table. The time base is derived from the processor 's bus clock, divided by a value specified in the divide configuration register. After reset, the timer is initialized to zero. The timer supports one-shot and periodic modes. The timer can be configured to interrupt the local processor with an arbitrary vector.

The library gets the system clock from the local APIC timer and auxiliary clock from either RTC or PIT channel 0 (define **PIT0_FOR_AUX** in the BSP). The macro **APIC_TIMER_CLOCK_HZ** must be defined to indicate the clock frequency of the local APIC timer.

This value is BSP-specific and is set in the BSP **configInum.h** file. The value for this clock rate can be found in the hardware documentation for your target board.

The parameters SYS_CLK_RATE_MIN, SYS_CLK_RATE_MAX, AUX_CLK_RATE_MIN, and AUX_CLK_RATE_MAX must be defined to provide parameter checking for the sysClkRateSet( ) and sysAuxClkRateSet( ) functions.

The timer driver uses the processor's on-chip TSC (timestamp counter) for the timestamp driver. The TSC is a 64-bit timestamp counter that is incremented every processor clock cycle. The counter is incremented even when the processor is halted by the HLT instruction or the external STPCLK# pin. The timestamp counter is set to 0 following a hardware reset of the processor. The RDTSC instruction reads the timestamp counter and is guaranteed to return a monotonically increasing unique value whenever executed, except for 64-bit counter wraparound. Intel guarantees, architecturally, that the timestamp counter frequency and configuration is such that it will not wraparound within 10 years after being reset to 0. The period for counter wrap is several thousands of years in P7 (Pentium 4), Atom, Core, Nehalem, and Intel-64 based processor families.

31. I/O Mapped Devices Reference

Several functions are available for I/O mapped devices.

sysInByte( )

Input one byte from I/O space.

sysOutByte( )

Output one byte to I/O space.

sysInWord( )

Input one word from I/O space.

sysOutWord( )

Output one word to I/O space.

sysInLong( )

Input one long word from I/O space.

sysOutLong( )

Output one long word to I/O space.

sysInWordString( )

Input a word string from I/O space.

sysOutWordString( )

Output a word string to I/O space.

sysInLongString( )

Input a long string from I/O space.

sysOutLongString( )

Output a long string to I/O space.

32. Memory-Mapped Devices

For memory-mapped devices, there are two kinds of memory protection provided by VxWorks: paging with the MMU and segmentation with the global descriptor table.

Because VxWorks operates at the highest processor privilege level, no “protection rings” exist.

Intel processors allow you to configure memory space into valid and invalid areas, even under supervisor mode. Thus, you receive a page fault only if the processor attempts to access addresses mapped as invalid, or addresses that have not been mapped. Conversely, if the processor attempts to access a nonexistent address space that has been mapped as valid, no page fault occurs.

Memory Considerations for VME (Intel 64)

If you are running VxWorks for Intel 64 on a VME board, be aware that addressing nonexistent memory or peripherals does not generate a bus error or fault.

33. PCI Bus

For supported PCI devices, see your BSP documentation.

34. Software Floating-Point Emulation

The software floating-point library is supported for IA-32 architectures that do not have on-chip FPUs; select INCLUDE_SW_FP for inclusion in the project facility VxWorks view to include the library in your system image.

> Note: The software floating-point library is not supported for Intel 64 in this release.

This library emulates each floating-point instruction by using the exception "Device Not Available". For other floating-point support information, see Supported Functions in mathALib on page 10.

35. Boot Configuration

VxWorks boot images read multiprocessor configuration data from a multiprocessor (MP) configuration table left in memory by the BIOS.

This table contains configuration information, such as interrupt routing and CPU count.

> Note: BIOS firmware is provided by third-party vendors, the quality of which is not under the control of Wind River Systems. While the MP table is usually correct, there are cases where there are errors in the table such as a failure to report the number of processors. In some cases, the table is missing entirely.

The following options are provided for IA-32 BSPs.

INCLUDE_MPTABLE_BOOT_OP

This option uses the BIOS generated MP table to obtain multiprocessor configuration data. This is the preferred method if the table left by the BIOS firmware is correct.

INCLUDE_USR_BOOT_OP

This option uses **staticMpApicData**, a static configuration table hard-coded by the user in **sysLib.c**, to obtain multiprocessor configuration data. An example structure is available in **sysLib.c** that can be used as a starting point but must be hand-edited for the target board.

INCLUDE_ACPI_BOOT_OP

This option builds internal tables for VxWorks using ACPI tables left in memory by the BIOS. If this option is selected, a code fragment can be automatically generated on the target shell for use to build a static table. This is printed to the boot shell using the vxMpTablePrint command. After capturing this code fragment, it can be pasted into **sysLib.c** and edited to accurately describe the target hardware. Once a correct table is built into the image, the **INCLUDE_USR_BOOT_OP** component can be used to boot the target.

For more details regarding the target boot sequence and options, see the readme file provided with your reference BSP. For example, *itl_generic_readme.md*.

36. BIOS E820 Memory Auto-size

The physical memory auto-size feature uses the BIOS enabled E820 system map query function.

During early boot ROM initialization, each successive execution of this BIOS function places the next memory address range identifier into a reserved memory location until the BIOS signals the final entry. Each memory address range descriptor contains a start address, a length, and a memory type. The memory types include available, reserved, ACPI NVS, and ACPI data.

During the VxWorks image initialization, this collection of identifiers is validated and entries marked as available are used to calculate the physical address corresponding to the top of usable memory. For 64-bit systems (and 32-bit systems supported by the **itl_generic** BSP), the available identifiers are used to populate the sysPhysRamDesc structure. Once a VxWorks image is booted, the vxBiosE820MapShow( ) function prints a summary of the physical RAM map to the terminal. This auto-size feature is enabled with the **INCLUDE_BIOS_E820_MEM_AUTOSIZE** component.

For complete documentation on E820 functionality, see the ACPI specification on the following website:

http://www.acpi.info/spec.htm

37. Expose the Target Memory Map Programmatically

During the VxWorks boot phase, the BSP interrogates the BIOS to automatically identify the memory regions that will be available to the running VxWorks kernel on this target once it has finished booting. The resulting memory layout, and the available memory regions, are exposed to the application or BSP or driver code using the VxWorks API function sysMemDescGet( ). This function is part of the Intel processor support library and returns a populated descriptor table that contains the running target's memory map.

For example:

```
    LOCAL PHYS_MEM_DESC sysPhysRamDesc[32];
    LOCAL STATUS success;

    success = sysMemDescGet(MEM_DESC_RAM, 0, sysPhysRamDesc);
```

Note:

You can also display the Intel target memory layout from the kernel shell.

```
-> sysMemDescShow()
```

38. Physical Addressing Support

Intel 64 systems are supported by the 64-bit VxWorks operating system. Therefore, it supports 64-bit virtual addressing.

For IA-32 systems, VxWorks is a 32-bit operating system therefore it supports 32-bit virtual addressing. However, some target hardware includes physical addressing space greater than 32 bits. Support for physical addressing space greater than 32 bits varies by architecture. The table below provides information on physical addressing for supported IA-32 architectures

Table 1. IA-32 Physical Addressing

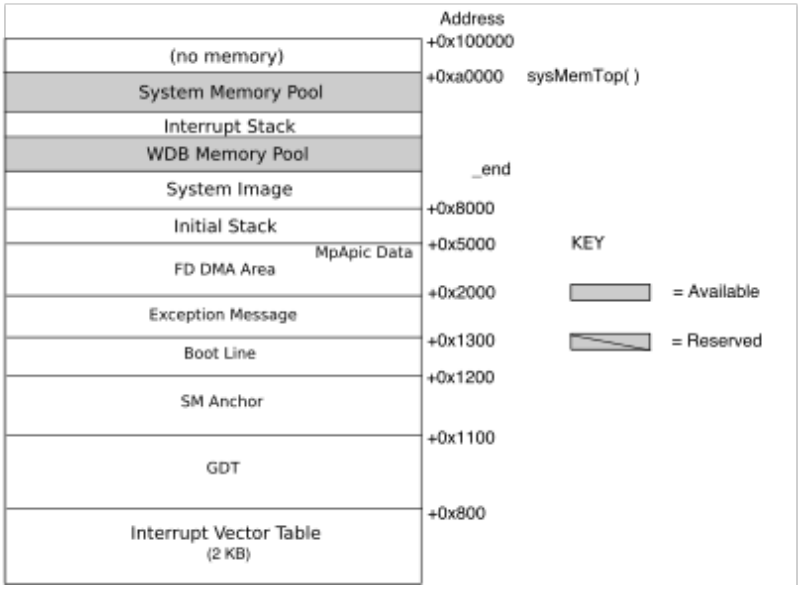
Architecture Variant	Virtual Addressing Size	Physical Addressing Size (PHYS_ADDR)	Supported PHYS_ADDR Bits
PENTIUM4	32 bit	32 bit	32 bit
ATOM	32 bit	32 bit	32 bit
CORE2	32 bit	32 bit	32 bit
NEHALEM	32 bit	32 bit	32 bit
SANDYBRIDGE	32 bit	32 bit	32 bit
HASWELL	32 bit	32 bit	32 bit

39. VxWorks Memory Layout for IA-32 Systems

Two memory layouts for IA-32 architectures are described in this section.

Lower Memory Option

Figure 3. VxWorks System Memory Layout (IA-32 Lower Memory)



The figures contain the following labels:

System Memory Pool

Size depends on size of system image and interrupt stack. The end of the free memory pool for this board is returned by sysMemTop( ).

Interrupt Stack

Size is defined by **ISR_STACK_SIZE** under **INCLUDE_KERNEL**. Location depends on system image size.

System Image

Entry point for VxWorks.

Initial Stack

Initial stack for usrInit( ), until usrRoot( ) gets allocated stack.

FD DMA Area

Diskette (floppy device) direct memory access area.

MpApic Data

APIC multiprocessor configuration table data area. (LOCAL_MEM_LOCAL_ADRS + 0x2000)

Exception Message

ASCII string of the fatal exception message.

Boot Line

ASCII string of boot parameters.

Global Descriptor Table (GDT)

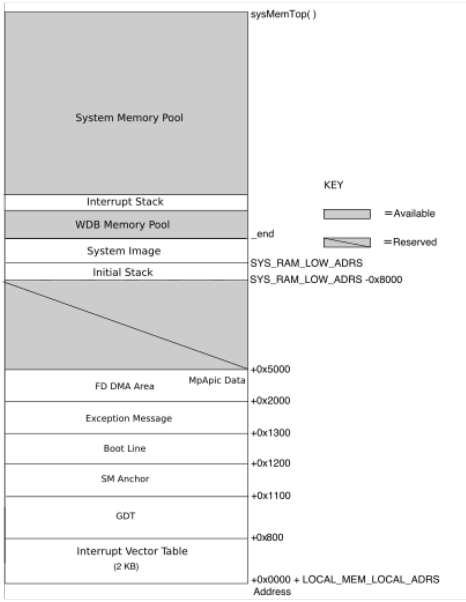
Anchor for the shared memory network (if there is shared memory on the board).

Interrupt Vector Table (IDT)

Table of exception/interrupt vectors (IDT).

Typical Upper Memory Configuration

Figure 4. VxWorks System Memory Layout (IA-32 Upper Memory)



In general, the boot image is placed in lower memory and the VxWorks image is placed in upper memory, leaving a gap between lower and upper memory. Some BSPs have additional configurations which must fit within their hardware constraints. For details, see the reference entry for each BSP.

40. VxWorks Memory Layout for Intel 64 Systems

Because the lower 2 gigabytes of physical memory is mapped to the upper 2 gigabytes of the virtual address space, and the lower 31 address bits are identical between these physical and virtual addresses, then the boot parameters, the persistent memory region, the system exception message region, and the user reserved memory region can all be accessed from both physical and virtual address space.

Access to the boot parameters is performed by the boot application code once it is executing in the upper 2 gigabytes of the virtual address space. The boot application code reads the boot parameters, and provides boot shell commands to edit the boot parameters. The boot application does not access persistent memory, the system exception message region, nor the user reserved memory.

On a warm start, the ROM code does not access the boot parameters memory region, the persistent memory region, the system exception message region, nor the user reserved memory region. The system start type is passed from romInit( ) to romStart( ), then to usrInit( ), which passes it to sysStart( ). The sysStart( ) function saves the system start type, warm or cold, into the global variable *sysStartType* . The following describes how the boot parameter, the persistent memory, the system exception messages, and the user reserved memory regions are cleared:

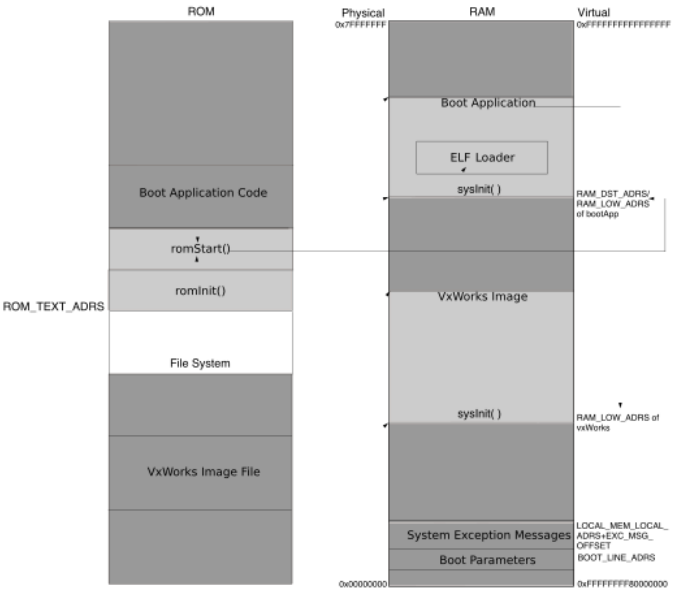
- Boot Parameter—The ROM code clears the boot parameters memory region on a cold start, by placing an EOS character at the beginning of the boot line string.
- Persistent Memory—It is the responsibility of the persistent memory system to check the state of the *sysStartType* variable to determine if the persistent memory region should be cleared on a cold start.
- System Exception Messages—In previous versions of VxWorks, the clearing of the system exception message region has been performed by the romStart( ) function on a cold start. However, the clearing of the system exception message region must to be moved from the romStart( ) function to the end of the sysStart( ) function after the sysMemDescInit( ) function call has completed.
- User Reserved Memory—The user needs to reference the *sysStartType* variable to decide whether to clear the user reserved memory region on a cold start or a warm start. The following describes the location in memory for the boot parameter, the persistent memory, and the system exception messages regions:
- Boot Parameter—The boot parameters memory region address is defined by the **BOOT_LINE_ADRS** macro.
- Persistent Memory—The persistent memory region location is determined by the sysMemDescInit( ) function.
- System Exception Messages—The system exception message region address is defined by sum of the **LOCAL_MEM_LOCAL_ADRS** and **EXC_MSG_OFFSET** macros.
- User Reserved Memory—The user reserved memory region location is determined by the sysMemDescInit( ) function.

Note that there are two separate **RAM_LOW_ADRS** values in the diagram; one for bootApp and another one for vxWorks. This is a side effect of using project builds. 64-bit only supports project builds. These different values are listed in a BSP's 00bsp.cdf file.

In the following example, there is a **RAM_LOW_ADRS** value for bootRam (**INCLUDE_BOOT_RAM_IMAGE**), bootApp (**INCLUDE_BOOT_APP**), and vxWorks (the last entry).

```
Parameter RAM_LOW_ADRS {
      NAME      Runtime kernel load address
      DEFAULT   INCLUDE_BOOT_RAM_IMAGE)::(0xffffffff91408000) \
      (INCLUDE_BOOT_APP)::(0xffffffff90008000) \
      0xffffffff80408000
}
```

Figure 5. Boot Application Loading and Executing a VxWorks Image



The boot code, the boot parameter memory, the persistent memory, and the loaded VxWorks image are all to be located in the 64-bit virtual address space. The boot code, the boot parameter memory, and the persistent memory all require an identity mapping between the physical memory address located in the 32-bit address space and the virtual address located in the 64-bit address space.

The boot code first executes romInit( ), which calls romStart( ). The romInit( ) code is responsible for setting up the mapping of the 64-bit address space for the boot code, the boot parameter memory, and the persistent memory. Both the romInit( ) and romStart( ) code execute in ROM. The romStart( ) code copies the rest of the boot code into RAM and executes it. The boot code is then executing in the 64-bit virtual address space.

The boot code executes the ELF loader code to load a VxWorks image file into the 64-bit address space, and returns the starting address in the 64-bit address space for this VxWorks image to the boot code. The ELF loader code reads the ELF image file to determine if it is a 32-bit image or a 64-bit image, and executes the appropriate ELF loader code for the image type. The ELF loader code reads the program header of the ELF image file to identify each of the loadable segments of the VxWorks image. The program header of the ELF file describes the starting physical address, the starting virtual address, and the size of each of the loadable segments. The ELF loader code then calls the VM library code to map memory space for the loadable segment into the 64-bit virtual address space. Once the memory for the loadable segment has been mapped into the 64-bit virtual address space, the ELF loader code reads the loadable segment into memory. Because the ELF loader code mapped the memory occupied by the VxWorks image file, then the boot code is able to execute the starting address of the VxWorks image file that was returned by the ELF loader code.

Loading and Executing 32-bit VxWorks from a 32-bit Boot ROM

A 64-bit boot ROM can load a 32-bit image. Support for the 64-bit boot ROM does not interfere with the ability of boot ROM code built for 32-bit VxWorks from loading and executing a 32-bit VxWorks image. No changes are required to maintain the support for loading and executing a 32-bit VxWorks image; building the 32-bit boot ROM with the 32-bit libraries is all that is required for a 32-bit BSP.

Note: A 32-bit boot ROM cannot load a 64-bit VxWorks image.

Loading and Executing 32-bit VxWorks from a 64-bit Boot ROM

Note: You must use an **itl_generic** boot application (bootApp) to boot the **itl_generic** BSP.

The 64-bit boot ROM, which is built from the 64-bit libraries, executes the ELF loader code to read the ELF header of the VxWorks image file to determine if the image type is an ELF-32 or ELF-64 image. It then executes the appropriate ELF loader code, either 32-bit or 64-bit. For 32-bit VxWorks images, the 32-bit ELF loader code is executed to load the 32-bit VxWorks image.

Intel architecture BSPs install a boot go override function for the boot application to execute. The boot go override function is executed after the VxWorks image has been loaded by the ELF loader. If the VxWorks image is an ELF32 image, then the boot go override function puts the CPU into 32-bit mode before executing image.

Note: The interrupt modes used in the VxWorks image and boot loader must be the same.

Loading and Executing 64-bit VxWorks from a 64-bit Boot ROM

The 64-bit boot ROM, which is built from the 64-bit libraries, executes the ELF loader code to read the ELF header of the VxWorks image file to determine if the image type is an ELF-32 or ELF-64 image. It then and execute the appropriate ELF loader code, either 32-bit or 64-bit. For 64-bit VxWorks images, the 64-bit ELF loader code is executed to load the 64-bit VxWorks image.

Note: The interrupt modes used in the VxWorks image and boot loader must be the same.

Mapping an Address Block into the MMU

The program header of the ELF file describes the starting physical address, the starting virtual address, and the size of each of the loadable segments. The address block for each loadable segment must be mapped by the MMU. The ELF loader code then calls the VM library code to map memory space for the loadable segment into the 64-bit virtual address space. Once the memory for the loadable segment has been mapped into the 64-bit virtual address space, the ELF loader code reads the loadable segment into memory.

The vmMap( ) function is used to resolve the mapping of addresses specified by the ELF file so that the boot ROM code can access these address locations. The vmMap( ) function requires that the physical and virtual addresses be on page boundaries, and the size of the block to be mapped must align to a page boundary. Therefore, additional code is implemented to ensure that page alignment occurs when calling vmMap( ).

Warm Booting from a 64-bit VxWorks Image

There are special considerations for warm boot from a 64-bit VxWorks image that is executing in virtual memory space back into the boot ROM. The boot ROM can be either a 64-bit or a 32-bit image. Because the boot ROM can be a 32-bit image, the sysToMonitor( ) code must put the CPU in 32-bit mode, and ensure that the boot ROM image is properly mapped to be accessible by the CPU to execute the boot ROM image. On Intel architecture BSPs, the sysToMonitor( ) function puts the CPU in 64-bit mode to run the boot loader in 64-bit mode.

41. Reference Material

Comprehensive information regarding IA-32 and Intel 64 hardware behavior and programming is beyond the scope of this document.

Intel Corporation provides several hardware and programming manuals for the IA-32 and Intel 64 processor families on its website:

http://www.intel.com/

Wind River recommends that you consult the hardware documentation for your processor or processor family as necessary during BSP development.